

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO THU THẬP
NỘI DUNG KHÓA HỌC UDEMY

Người hướng dẫn:

Kiran Nayak

Ban chủ nhiệm câu lạc bộ CEEC

Sinh viên thực hiện:

1. Huỳnh Thanh Sang

23521341

TP. Hồ Chí Minh, 6/2024

[illegible]

Người nhận xét

1

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành tới các thành viên ban chủ nhiệm câu lạc bộ CEEC – Computer Engineering Embedded Club trực thuộc khoa Kỹ thuật Máy tính và thầy Kiran Nayak – Giảng viên khóa học “Mastering Microcontroller and Embedded Driver Development” đã giúp em bước đầu hoàn thành kiến thức và kỹ năng căn bản cần có để hoàn thành tìm hiểu về quá trình làm chủ và xây dựng driver cho đa dạng bộ vi điều khiển.

Tuy nhiên trong quá trình học hỏi và ghi chú khóa học, do kiến thức chuyên ngành và yếu tố kỹ thuật còn hạn chế nên em vẫn còn nhiều sai sót khi tìm hiểu, đánh giá và trình bày chi tiết về phần nội dung khóa học. Rất mong nhận được sự quan tâm, góp ý của các thầy/ cô giảng viên và các thành viên của ban chủ nhiệm câu lạc bộ để bài cáo cáo em được đầy đủ và hoàn thiện hơn.

Xin chân thành cảm ơn.

MỤC LỤC

LỜI CẢM ƠN.....	2
MỤC LỤC	3
MỤC LỤC HÌNH ẢNH.....	6
MỤC LỤC BẢNG	7
LỜI MỞ ĐẦU.....	8
CHƯƠNG 1. GIỚI THIỆU YÊU CẦU	9
1. Tên báo cáo:.....	9
2. Mục đích báo cáo:	9
3. Nội dung báo cáo tổng quan:	9
CHƯƠNG 2. NỘI DUNG BÁO CÁO	10
1. Giới thiệu về bộ vi xử lý <i>ARM Cortex-M3</i> :.....	10
1.1. Các đặc điểm nổi bật của bộ vi xử lý:.....	10
1.2. Sơ đồ khối hoạt động của bộ vi xử lý:.....	11
2. Kiến trúc bộ vi điều khiển <i>STM32F103C8T6</i> :.....	15
2.1. Đối với thiết bị có mật độ thiết kế thấp – trung bình - cao:	15
2.2. Đối với thiết bị dùng cho đường dẫn kết nối:.....	16
2.3. Đường dẫn truyền ICode:	17
2.4. Đường dẫn truyền DCode:	15
2.5. Đường dẫn hệ thống:.....	17
2.6. Đường dẫn truyền DMA:.....	17
2.7. Cầu dẫn AHB/APB:	18
3. Phân vùng bộ nhớ và phạm vi truy cập ngoại vi:.....	18
3.1. Bộ nhớ FLASH:	19
3.2. Bộ nhớ SRAM:	20
3.3. Phạm vi truy cập ngoại vi:.....	21
4. Tìm hiểu về thiết kế cây xung nhịp:.....	21
4.1. Bộ xung nhịp dao động HSE:	23

4.2.	Bộ xung nhịp dao động HSI:.....	24
4.3.	Bộ xung nhịp lặp khóa pha PLL:.....	24
4.4.	Tùy chọn xung nhịp hệ thống:.....	24
4.5.	Hệ thống bảo vệ xung nhịp:.....	24
5.	Cơ chế khởi động chung đối với các ngoại vi:.....	25
6.	Các thiết lập đồng bộ chung khi lập trình:.....	25
6.1.	Từ khóa volatile:.....	25
6.2.	Các thiết lập khác:.....	26
7.	Xử lý thanh ghi và cơ chế lập trình đối với ngoại vi RCC:.....	26
7.1.	Xử lý thanh ghi:.....	26
7.2.	Cơ chế lập trình:.....	29
8.	Giải mã mạch và cơ chế hoạt động của ngoại vi GPIO:.....	31
8.1.	Giải mã mạch:.....	31
8.2.	Cơ chế hoạt động nếu được thiết lập là đầu vào:.....	32
8.3.	Cơ chế hoạt động nếu được thiết lập là đầu ra:.....	33
8.4.	Cơ chế hoạt động nếu được thiết lập là chức năng thay thế:.....	34
9.	Xử lý thanh ghi và cơ chế lập trình đối với ngoại vi GPIO:.....	35
9.1.	Xử lý thanh ghi:.....	35
9.2.	Cơ chế lập trình:.....	37
10.	Xử lý tạo lập trình điều khiển ngoại vi GPIO:.....	39
11.	Làm việc với bảng mã Vector:.....	45
12.	Tìm hiểu về thiết kế ngắt luồng ngoài:.....	46
13.	Xử lý thanh ghi ngắt luồng ngoài trên GPIO:.....	47
13.1.	Thanh ghi EXTI_IMR:.....	48
13.2.	Thanh ghi EXTI_EMR:.....	48
13.3.	Thanh ghi EXTI_RSTR:.....	48
13.4.	Thanh ghi EXTI_FTSR:.....	48
13.5.	Thanh ghi EXTI_SWIER:.....	49
13.6.	Thanh ghi EXTI_PR:.....	49
14.	Xử lý thanh ghi bộ điều khiển ngắt chồng luồng Vector:.....	49

14.1.	Thanh ghi NVIC_ISERx (x từ 0 tới 2):	50
14.2.	Thanh ghi NVIC_ICERx (x từ 0 tới 2):	50
14.3.	Thanh ghi NVIC_ISPRx (x từ 0 tới 2):	51
14.4.	Thanh ghi NVIC_ICPRx (x từ 0 tới 2):	51
14.5.	Thanh ghi NVIC_IPRx (x từ 0 tới 21):	51
15.	Cơ chế xử lý ngắt luồng đối với ngoại vi GPIO:	52
15.1.	Ở phía ngoại vi:	52
15.2.	Ở phía NVIC:	53
16.	Tìm hiểu cấu hình xử lý ngắt luồng cho ngoại vi GPIO:	53
17.	Kiểm định bằng các chương trình mẫu:	56
17.1.	Điều khiển LED bằng nút nhấn ngoài có nối trở 10kOhm:	56
17.2.	Điều khiển LED bằng nút nhấn ngoài không có trở và dùng ngắt ngoài: ...	59
CHƯƠNG 3: KẾT LUẬN		60
TÀI LIỆU THAM KHẢO		60

MỤC LỤC HÌNH ẢNH

Hình 1: Sơ đồ khởi hoạt động của bộ vi xử lý ARM Cortex-M3.....	11
Hình 2: Bus Matrix đối với thiết bị có mật độ thiết kế thấp - trung - cao.....	15
Hình 3: Hệ thống kết nối ở thiết bị có mật độ thiết kế thấp - trung - cao.....	16
Hình 4: Bus Matrix đối với thiết bị dùng cho đường dẫn kết nối.....	16
Hình 5: Hệ thống kết nối đối với thiết bị dùng cho đường dẫn kết nối.....	17
Hình 6: Cây xung nhịp trên STM32F103C8T6.....	23
Hình 7: Cấu trúc thanh ghi RCC_CFGR.....	27
Hình 8: Cấu trúc thanh ghi RCC_APB2RSTR.....	27
Hình 9: Cấu trúc thanh ghi RCC_APB1RSTR.....	28
Hình 10: Cấu trúc thanh ghi RCC_AHBENR.....	28
Hình 11: Cấu trúc thanh ghi RCC_APB2ENR.....	28
Hình 12: Cấu trúc thanh ghi RCC_APB1ENR.....	29
Hình 13: Cấu trúc cơ bản chân GPIO.....	32
Hình 14: Chân GPIO đầu vào.....	33
Hình 15: Chân GPIO đầu ra.....	34
Hình 16: Chân GPIO tín hiệu Analog.....	34
Hình 17: Cấu trúc thanh ghi GPIOx_CRL.....	35
Hình 18: Cấu trúc thanh ghi GPIOx_CRH.....	36
Hình 19: Cấu trúc thanh ghi GPIOx_IDR.....	36
Hình 20: Cấu trúc thanh ghi GPIOx_ODR.....	36
Hình 21: Cấu trúc thanh ghi GPIOx_BSRR.....	37
Hình 22: Bảng Vector tóm tắt.....	45
Hình 23: Sơ đồ ngắt luồng ngoài - EXTI.....	46
Hình 24: Cấu trúc thanh ghi EXTI_IMR.....	48
Hình 25: Cấu trúc thanh ghi EXTI_EMR.....	48
Hình 26: Cấu trúc thanh ghi EXTI_RTSTR.....	48
Hình 27: Cấu trúc thanh ghi EXTI_FTSR.....	48
Hình 28: Cấu trúc thanh ghi EXTI_SWIER.....	49
Hình 29: Cấu trúc thanh ghi EXTI_PR.....	49
Hình 30: Cấu trúc thanh ghi NVIC_ISERx.....	50
Hình 31: Cấu trúc thanh ghi NVIC_ICERx.....	50
Hình 32: Cấu trúc thanh ghi NVIC_ISPRx.....	51
Hình 33: Cấu trúc thanh ghi NVIC_ICPRx.....	51
Hình 34: Cấu trúc thanh ghi NVIC_IPRx.....	52
Hình 35: Sơ đồ kết nối chương trình 1.....	57
Hình 36: Sơ đồ kết nối chương trình 2.....	59

MỤC LỤC BẢNG

Bảng 1: Thu gọn từ khóa volatile.....	25
Bảng 2: Định nghĩa các mức giá trị 0/1.....	26
Bảng 3: Định nghĩa địa chỉ và biến đại diện RCC.	29
Bảng 4: Cấu trúc hóa bộ thanh ghi cho ngoại vi RCC.....	30
Bảng 5: Quy tắc sử dụng truy xuất thanh ghi.	30
Bảng 6: Quy tắc set giá trị cho ô bit trên thanh ghi.....	30
Bảng 7: Bảng cấu hình bit CNF, bit MODE và ODRx.	31
Bảng 8: Bảng cấu hình bit MODE chuyên sâu.....	32
Bảng 9: Công thức truy cập bit MODE.	35
Bảng 10: Công thức truy cập bit CNF.	35
Bảng 11: Công thức chuyển đổi pin.....	36
Bảng 12: Định nghĩa địa chỉ và biến đại diện cho GPIO.	37
Bảng 13: Cấu trúc bộ thanh ghi cho GPIO.....	38
Bảng 14: Định nghĩa macro cho khởi động và vô hiệu xung clock cho GPIO.	38
Bảng 15: Cấu trúc hóa các thành phần sử dụng trong tương tác với GPIO.....	39
Bảng 16: đoạn mã GPIO_Clock_Control.....	40
Bảng 17: đoạn mã GPIO_Init.	42
Bảng 18: đoạn mã GPIO_De_Init.	43
Bảng 19: đoạn mã GPIOx_PERI_CLK_RESET().	43
Bảng 20: đoạn mã GPIO_Read_From_Pin.	43
Bảng 21: đoạn mã GPIO_Read_From_Port.....	44
Bảng 22: đoạn mã GPIO_Write_To_Pin.....	44
Bảng 23: đoạn mã GPIO_Write_To_Port.	44
Bảng 24: đoạn mã GPIO_Toggle_Pin.	45
Bảng 25: Công thức thiết lập giá trị với số IRQ trong NVIC_ISER1.	50
Bảng 26: Công thức thiết lập giá trị với số IRQ trong NVIC_ISER2.	50
Bảng 27: Công thức tính số thứ tự thanh ghi.	52
Bảng 28: Công thức tính độ dời vị trí truy cập.....	52
Bảng 29: Công thức truy cập cuối cùng.	52
Bảng 30: Mở rộng hàm GPIO_Init.....	54
Bảng 31: đoạn mã hàm GPIO_IRQ_Interrupt_Config.....	55
Bảng 32: đoạn mã hàm GPIO_IRQ_Number_Config.....	55
Bảng 33: đoạn mã hàm GPIO_IRQ_Handler.	56
Bảng 34: Ví dụ override hàm xử lý ISR.	56
Bảng 35: đoạn mã cấu hình chương trình 1.	58
Bảng 36: đoạn mã thuật toán chương trình 1.....	58
Bảng 37: đoạn mã thuật toán chương trình 2.....	60

LỜI MỞ ĐẦU

Lập trình nhúng là một hướng đi từ lâu song hành cùng nhóm ngành vi mạch và điện – điện tử, giúp con người xây dựng và điều khiển mạch theo các chức năng đề ra. Trong đó, quá trình tìm hiểu về tài liệu và xây dựng Driver cho bộ vi điều khiển là công đoạn tốn nhiều thời gian và công sức của người lập trình viên.

Sau một thời gian tìm hiểu và nhận được sự hướng dẫn của các anh chị trong câu lạc bộ CEEC – Computer Engineering Embedded Club và thầy Kiran Nayak, em đã nỗ lực xây dựng trình điều khiển nhúng cho phép người dùng hoàn thiện đầy đủ. Tuy nhiên, với kinh nghiệm và thời gian thực hiện còn hạn hẹp nên không thể tránh khỏi các sai sót. Em rất mong nhận được sự giúp đỡ và tham khảo ý kiến từ các anh chị cùng các bạn để có thể hoàn thiện báo cáo.

CHƯƠNG 1. GIỚI THIỆU YÊU CẦU

1. Tên báo cáo:

- Tổng quan về quá trình làm chủ và xây dựng trình điều khiển ngoại vi GPIO cho bộ vi điều khiển STM32.

2. Mục đích báo cáo:

- Cung cấp cơ chế hoạt động của bộ vi điều khiển STM32 đối với các ngoại vi GPIO.
- Mô tả quy trình xây dựng trình điều khiển cho ngoại vi và các lưu ý.
- Hỗ trợ nguồn dữ liệu cho buổi Seminar về các khóa học.

3. Nội dung báo cáo tổng quan:

- Giới thiệu về bộ vi xử lý *ARM Cortex-M3*.
- Kiến trúc bộ vi điều khiển STM32F103C8T6.
- Phân vùng bộ nhớ và phạm vi truy cập ngoại vi.
- Tìm hiểu về thiết kế xung Clock của bộ vi điều khiển.
- Cơ chế khởi động chung đối với các ngoại vi.
- Xử lý thanh ghi và cơ chế lập trình đối với ngoại vi RCC.
- Giải mã mạch và cơ chế hoạt động của ngoại vi GPIO.
- Xử lý thanh ghi và cơ chế lập trình đối với ngoại vi GPIO.
- Xử lý tạo lập trình điều khiển ngoại vi GPIO.
- Làm việc với bảng mã Vector.
- Tìm hiểu về thiết kế ngắt luồng, bộ điều khiển ngắt chồng luồng đã được Vector hóa.
- Cơ chế xử lý ngắt luồng.
- Tìm hiểu cấu hình xử lý ngắt luồng cho ngoại vi GPIO.

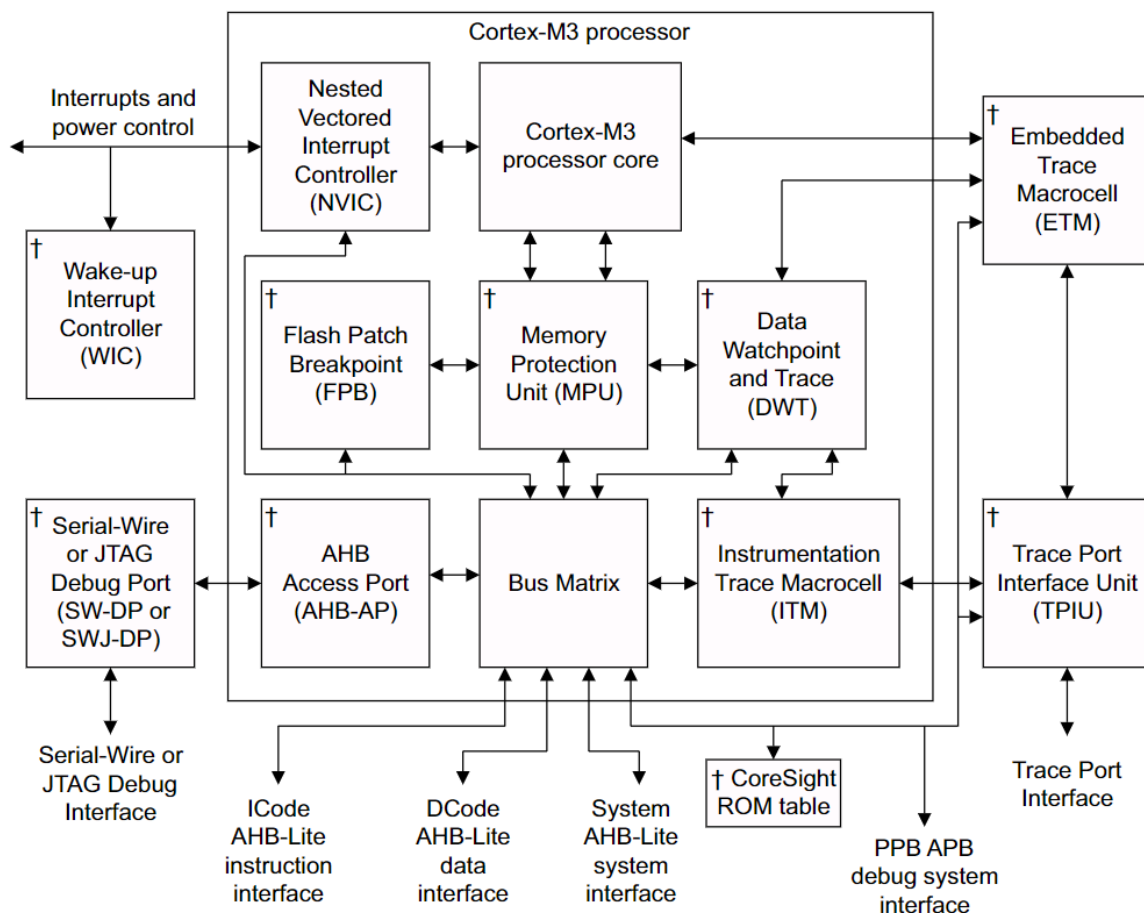
CHƯƠNG 2. NỘI DUNG BÁO CÁO

1. Giới thiệu về bộ vi xử lý *ARM Cortex-M3*:

1.1. Các đặc điểm nổi bật của bộ vi xử lý:

- Kiến trúc *ARMv7-M*.
- Bốn giao thức dẫn truyền dữ liệu rút gọn ưu việt tốc độ cao (*Advanced High-performance Bus – AHB) – Lite*.
- Bổ sung tập lệnh con của *Thumb/Thumb-2*.
- Pipeline 3 tầng.
- Bộ điều khiển ngắt chồng luồng đã được Vector hóa (*Nested Vectored Interrupt Controller – NVIC*).
- 8 phân vùng bảo vệ bộ nhớ (*Memory Protection Unit – MPU*) tùy chọn kèm theo các phân vùng đệm con và phân vùng nền.
- Tập lệnh xử lý trường Bit tích hợp (*Integrated Bit-field Processing Instructions*).
- Thao tác Bit ở cấp độ dẫn truyền dữ liệu (*Bus Level Bit Banding*).

1.2. Sơ đồ khối hoạt động của bộ vi xử lý:



Hình 1: Sơ đồ khối hoạt động của bộ vi xử lý ARM Cortex-M3.

- Trong sơ đồ, khối vi xử lý thực tế chỉ chiếm 1 phần nhỏ trong tổng thể kiến trúc mà thay vào đó sẽ được tích hợp các khối chức năng chính khác như *NVIC*, *MPU*, *ITM*, *AHB-AP*,... Điều này giúp thu nhỏ bộ vi xử lý đồng thời cho phép tối ưu hiệu suất khi có công truy cập đường dẫn truyền dữ liệu *AHB*.

1.2.1. Khối điều khiển ngắt chồng luồng đã được Vector hóa:

Khối điều khiển ngắt chồng luồng đã được Vector hóa được tích hợp ngay bên trong nhân xử lý để tối thiểu giảm độ trễ trong xử lý ngắt luồng. Trong đó, khối được tích hợp các chức năng sau:

- Cho phép ngắt luồng ngoại tuyến có thể cấu hình từ 1 đến 240.

- Hỗ trợ bit ưu tiên có thể cấu hình từ 3 tới 8.
- Linh hoạt điều chỉnh mức ưu tiên cho đa ngắt luồng.
- Hỗ trợ liên kết chuyển và xử lý trễ cho ngắt luồng để cho phép tránh quá tải xử lý lưu trữ và khôi phục trạng thái của các ngắt luồng.
- Tự động lưu trữ trạng thái của vi xử lý ở đầu vào và khôi phục ở đầu ra của ngắt luồng tránh quá tải xử lý.
- Hỗ trợ điều khiển đánh thức ngắt luồng, cung cấp chế độ ngủ sâu với mức tiêu thụ năng lượng cực thấp.

1.2.2. Khối bảo vệ bộ nhớ:

Khối bảo vệ bộ nhớ là khối tùy chọn giúp tránh lỗi bộ nhớ và thất thoát dữ liệu được xây dựng với các đặc điểm sau:

- Được hỗ trợ tám phân vùng bộ nhớ.
- Cho phép sử dụng hiệu quả các phân vùng bộ nhớ với tính năng vô hiệu phân vùng con (*Sub Region Disable – SRD*).
- Khả năng tạo ra phân vùng nền sao chép các thuộc tính của bản đồ bộ nhớ mặc định.

1.2.3. Giao thức dẫn truyền và giám sát dữ liệu:

- Gồm các khối ma trận dẫn truyền dữ liệu (*Bus Matrix – BM*), khối truy cập đường dẫn AHB (*AHB Access Port – AHB-AP*), khối *Macrocell* truy vết sự kiện (*Instrumentation Trace Macrocell – ITM*), khối *Macrocell* truy vết lệnh (*Embedded Trace Macrocell – ETM*) và khối truy vết cổng giao thức (*Trace Port Interface Unit – TPIU*). Trong đó, hai khối gồm *BM* và *AHB-AP* là hai khối được sử dụng nhiều nhất cho đa tác vụ liên quan đến xử lý dữ liệu, còn lại các khối như *ITM*, *ETM* và *TPIU* sẽ không được đề cập trong phần báo cáo này.

1.2.3.1. Khối ma trận dẫn truyền dữ liệu (*Bus Matrix – BM*):

- Khối ma trận dẫn truyền dữ liệu (*Bus Matrix – BM*) quản lý các thứ tự truy cập dữ liệu giữa các đường dẫn dữ liệu cốt lõi của hệ thống và đường dẫn chủ thể.

- Truyền/truy xuất dữ liệu trực tiếp (*Direct memory access – DMA*) thông qua thuật toán *Round Robin*.
- Trong đó, thuật toán *Round Robin* về mặt tư tưởng sẽ áp dụng 1 hàng đợi vòng (*Circular Queue*) cho từng tiến trình cần sử dụng *Bus Matrix* với 1 mức thời gian được bộ vi xử lý phân bổ gọi là *Time Quantum* (giá trị dao động từ 10 tới 100ms, áp dụng cho mọi chu trình). Với một tiến trình nào đã sử dụng xong thì sẽ được đẩy ra khỏi hàng đợi vòng và nhường chỗ cho tiến trình kế tiếp. Nếu một tiến trình mới được ghi nhận thì sẽ được thêm vào cuối dãy theo chỉ mục tiến trình được đánh dấu.
- Ở các thiết bị dùng để kết nối, *Bus Matrix* sẽ gồm 5 đường dẫn cha gồm *CPU DCode*, *System Bus*, *Ethernet DMA*, *DMA1* và *DMA2 Bus* và 3 đường dẫn con gồm *FLITF*, *SRAM* và cầu dẫn *AHB2APB*.
- Đối với các thiết bị khác, *Bus Matrix* sẽ cho phép cung cấp 4 đường dẫn cha gồm *CPU DCode*, *System Bus*, *DMA1* và *DMA2 Bus* và 4 đường dẫn con gồm *FLITF*, *SRAM* và cầu dẫn *AHB2APB*.

1.2.3.2. Giao thức bộ nhớ ICode:

- Giao thức bộ nhớ *ICode* là giao thức truyền lệnh từ bộ nhớ mã lệnh kéo dài trong khoảng 0x00000000 tới 0x1FFFFFFF, nằm trên đường dẫn truyền dữ liệu *AHB-Lite* 32-bit.
- Bộ gỡ lỗi (*Debugger*) không được phép truy cập vào giao thức này.
- Các truy xuất và giao tiếp với lệnh đều có độ rộng ở mức *Word* (tương ứng 32-bit). Tổng số lệnh truy xuất trên 1 *Word* phụ thuộc vào mã lệnh tức thời và vị trí sắp xếp trong bộ nhớ.

1.2.3.3. Giao thức bộ nhớ DCode:

- Giao thức bộ nhớ *DCode* là giao thức truyền dữ liệu và lệnh thuộc bộ gỡ lỗi (*Debugger*) từ bộ nhớ mã lệnh kéo dài trong khoảng 0x00000000 tới 0x1FFFFFFF, nằm trên đường dẫn truyền dữ liệu *AHB-Lite* 32-bit.

- Quyền truy cập vào dữ liệu có mức ưu tiên cao hơn so với quyền truy cập vào lệnh thuộc bộ gỡ lỗi. Nghĩa là quyền truy cập vào lệnh thuộc bộ gỡ lỗi chỉ được thông qua khi quyền truy cập vào dữ liệu trên đa nhân xử lý hoàn tất.
- Bộ kiểm soát logic ở giao thức này cho phép chuyển đổi các dữ liệu chưa được sắp xếp đúng và quyền truy cập gỡ lỗi thành hai hoặc ba phân quyền truy cập có sắp xếp chuẩn phụ thuộc vào kích thước và sự sắp xếp của phần chưa được sắp xếp chuẩn.

1.2.3.4. Giao thức nội bộ:

- Giao thức nội bộ hay *System Interface* là giao thức hỗ trợ truyền lệnh, truy cập dữ liệu và gỡ lỗi từ khoảng địa chỉ thứ nhất là 0x20000000 tới 0xDFFFFFFF và khoảng địa chỉ thứ 2 là 0xE0100000 tới 0xFFFFFFFF, nằm trên đường dẫn truyền dữ liệu *AHB-Lite* 32-bit.
- Quyền truy cập được xác định theo thứ tự như sau:
 - o Dữ liệu.
 - o Lệnh.
 - o Lệnh gỡ lỗi.
- Giao thức nội bộ đồng thời được tích hợp bộ điều khiển logic để xử lý các truy cập không hợp lệ, truy cập di dời điểm ngắt phần cứng (Flash Patch Breakpoint – FPB) và truyền lệnh trong Pipeline.

1.2.3.5. Đường dẫn truyền ngoại vi nội tuyến:

- Đường dẫn truyền ngoại vi nội tuyến hay *Private Peripheral Bus – PPB* cho phép truy cập dữ liệu và gỡ lỗi trong phạm vi bộ nhớ ngoài *PPB* từ 0xE0040000 tới 0xE00FFFFF.
- Đường dẫn này và các ngoại vi được *ARM* phát triển đều nằm chung trên đường dẫn truyền dữ liệu *APB* 32-bit.
- Quyền truy cập vào dữ liệu có mức ưu tiên cao hơn so với quyền truy cập vào lệnh thuộc bộ gỡ lỗi. Nghĩa là quyền truy cập vào lệnh thuộc bộ gỡ lỗi chỉ được thông qua khi quyền truy cập vào dữ liệu trên đa nhân xử lý hoàn tất.

- Chỉ có 1 số địa chỉ bit cần thiết để giải mã phân vùng bộ nhớ ngoài *PPB* được hỗ trợ trên đường dẫn này.

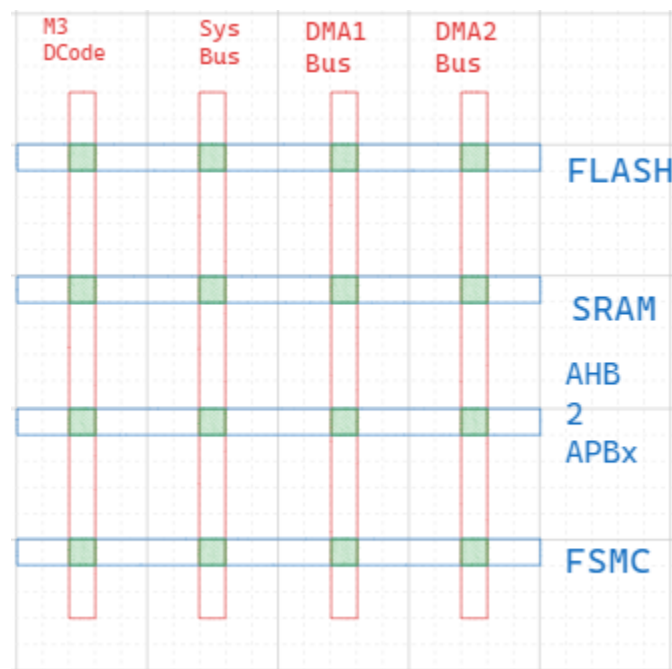
2. Kiến trúc bộ vi điều khiển STM32F103C8T6:

2.1. Đường dẫn truyền DCode:

- Giúp kết nối đường dẫn truyền tải và truy cập gỡ lỗi (*Literal Load and Debug Access* được thay thế ngắn gọn thành *Debug Bus – D-Bus*) của bộ vi xử lý tới giao thức lệnh của bộ nhớ *FLASH*.

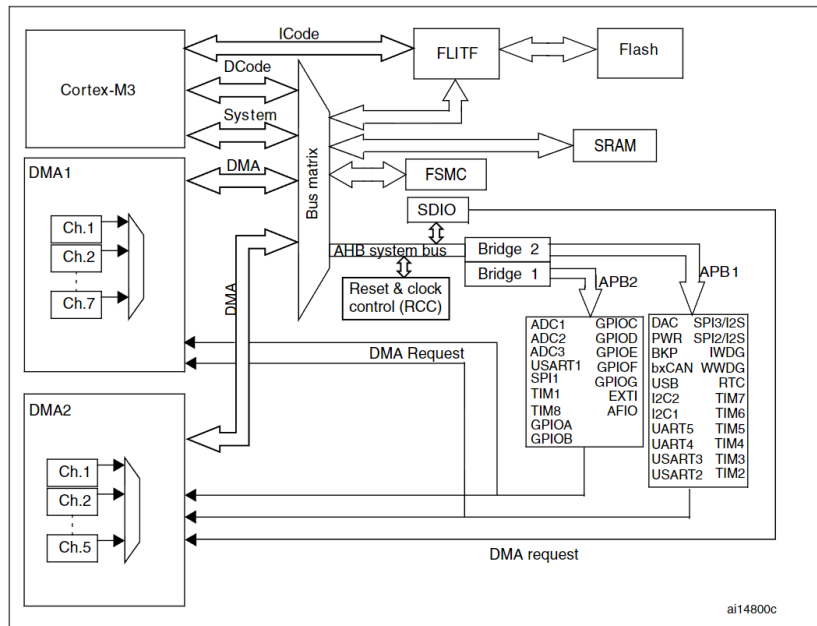
2.2. Đối với thiết bị có mật độ thiết kế thấp – trung bình - cao:

- Hệ thống lúc này gồm có bốn *master* và bốn *slave*, trong đó:
 - o Bốn *master* lần lượt là *DCode (D-bus)*, *Bus hệ thống (System Bus – S-bus)*, *GP-DMA1 & 2*.
 - o Bốn *slave* lần lượt là *SRAM*, *FLASH*, *FSMC* và cầu dẫn *AHB2APB*.



Hình 2: Bus Matrix đối với thiết bị có mật độ thiết kế thấp - trung - cao.

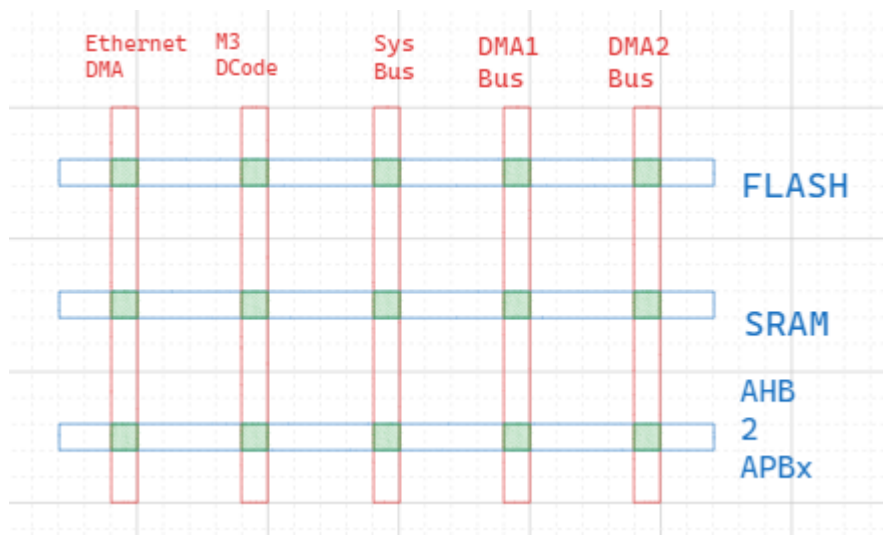
- Hệ thống được kết nối liên thông sử dụng kiến trúc đường dẫn *AHB* đa tầng như hình 2:



Hình 3: Hệ thống kết nối ở thiết bị có mật độ thiết kế thấp - trung - cao.

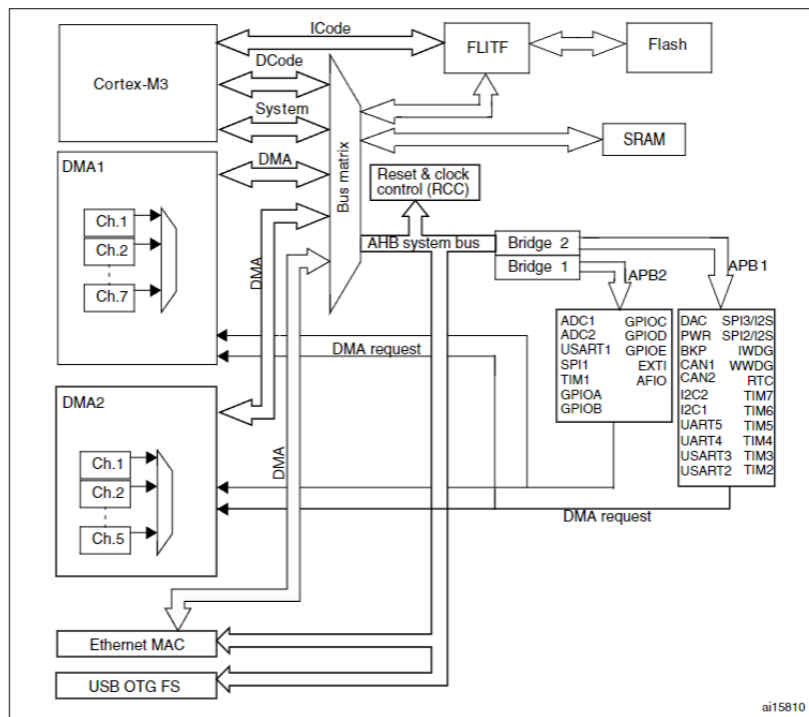
2.3. Đối với thiết bị dùng cho đường dẫn kết nối:

- Hệ thống lúc này gồm có năm *master* và ba *slave*, trong đó:
 - Năm master lần lượt là *DCode* (*D-bus*), Bus hệ thống (*System Bus – S-bus*), *GP-DMA1 & 2* và *Ethernet DMA*.
 - Ba slave lần lượt là *SRAM*, *FLASH* và cầu dẫn *AHB2APB*.



Hình 4: Bus Matrix đối với thiết bị dùng cho đường dẫn kết nối.

- Hệ thống được kết nối liên thông sử dụng kiến trúc đường dẫn *AHB* đa tầng như hình 3:



Hình 5: Hệ thống kết nối đối với thiết bị dùng cho đường dẫn kết nối.

2.4. Đường dẫn truyền ICode:

- Giúp kết nối đường dẫn truyền lệnh (*Instruction Bus – I-Bus*) của bộ vi xử lý tới giao thức lệnh của bộ nhớ *FLASH*.

2.5. Đường dẫn hệ thống:

- Giúp kết nối đường dẫn ngoại vi nội bộ của vi xử lý tới *Bus Matrix*, nơi quản lý các thứ tự truy cập dữ liệu giữa các đường dẫn dữ liệu cốt lõi của hệ thống và đường dẫn chủ thể.

2.6. Đường dẫn truyền DMA:

- Giúp kết nối giao thức chủ thể *AHB* của *DMA* tới *Bus Matrix*.

2.7. Cầu dẫn AHB/APB:

- Cung cấp kết nối đồng bộ giữa đường dẫn *AHB* và 2 đường dẫn *APB*. Trong đó, *APB1* bị giới hạn hoạt động ở tần số 36MHz còn *APB2* thì được hoạt động ở công suất tối đa của vi xử lý.
- Trước khi mỗi lần sử dụng ngoại vi của thiết bị thì phải hoạt hóa xung clock của từng ngoại vi thông qua thanh ghi *RCC_AHBENR*, *RCC_APB2ENR* hoặc *RCC_APB1ENR*.
- Sau mỗi lần reset thiết bị thì các xung clock của ngoại vi đều bị vô hiệu.

3. Phân vùng bộ nhớ và phạm vi truy cập ngoại vi:

- Phân vùng bộ nhớ được thành các phần như trong hình 4:

- Khối bộ nhớ chính có đa dạng kích thước và được chia thành các trang Kbytes.
- Khối thông tin gồm 2360×64 bit cho thiết bị có đường dẫn kết nối và 258×64 bit cho các loại thiết bị khác.
- Được tích hợp giao thức *FLITF* với các tính năng:
 - Đọc giao thức với bộ đệm đọc trước (*prefetch buffer* 2×64 bit *Word*, trong đó 1 *Word* ứng với 32-bit.).
 - Bộ tải *byte* tùy chọn.
 - Thao tác lập trình và xóa.
 - Bảo mật đọc và ghi dữ liệu.
- Khoảng địa chỉ từ $0x08000000$ tới $0x0801FFFF$.

3.2. Bộ nhớ SRAM:

- Là loại bộ nhớ tĩnh 64 Kbytes, cho phép truy cập loại *Byte*, *Half-Word* và *Word*.
- Địa chỉ mở đầu của bộ nhớ là $0x20000000$.

3.3. Phạm vi truy cập ngoại vi:

Boundary address	Peripheral	Bus	Register map	Boundary address	Peripheral	Bus	Register map
0x5000 0000 - 0x5000 03FF	USB OTG FS	AHB	Section 26.14.6 on page 778	0x4000 7800 - 0x4000 FFFF	Reserved	APB1	
0x4003 0000 - 0x4FFF FFFF	Reserved			0x4000 7400 - 0x4000 77FF	DAC		Section 12.5.14 on page 252
0x4002 8000 - 0x4002 9FFF	Ethernet		Section 27.8.5 on page 946	0x4000 7000 - 0x4000 73FF	Power control PWR		Section 4.4.3 on page 65
0x4002 3400 - 0x4002 7FFF	Reserved	AHB		0x4000 6C00 - 0x4000 6FFF	Backup registers (BKP)	APB1	Section 5.4.5 on page 70
0x4002 3000 - 0x4002 33FF	CRC		Section 3.4.4 on page 52	0x4000 6800 - 0x4000 6BFF	Reserved		
0x4002 2000 - 0x4002 23FF	Flash memory interface			0x4000 6400 - 0x4000 67FF	bxCAN1		Section 22.9.5 on page 583
0x4002 1400 - 0x4002 1FFF	Reserved			0x4000 6800 - 0x4000 6BFF	bxCAN2	APB1	Section 22.9.5 on page 583
0x4002 1000 - 0x4002 13FF	Reset and clock control RCC		Section 6.3.11 on page 102	0x4000 6000 ⁽¹⁾ - 0x4000 63FF	Shared USB/CAN SRAM 512 bytes		
0x4002 0800 - 0x4002 0FFF	Reserved			0x4000 5C00 - 0x4000 5FFF	USB device FS registers		Section 21.5.4 on page 540
0x4002 0400 - 0x4002 07FF	DMA2		Section 10.4.7 on page 196	0x4000 5800 - 0x4000 5BFF	I2C2	APB1	Section 24.6.10 on page 652
0x4002 0000 - 0x4002 03FF	DMA1		Section 10.4.7 on page 196	0x4000 5400 - 0x4000 57FF	I2C1		Section 24.6.10 on page 652
0x4001 8400 - 0x4001 7FFF	Reserved			0x4000 5000 - 0x4000 53FF	UART5		Section 25.6.8 on page 693
0x4001 8000 - 0x4001 83FF	SDIO		Section 20.9.16 on page 510	0x4000 4C00 - 0x4000 4FFF	UART4	APB1	Section 25.6.8 on page 693
0x4001 4000 - 0x4001 7FFF	Reserved	APB2		0x4000 4800 - 0x4000 4BFF	USART3		Section 25.6.8 on page 693
0x4001 3C00 - 0x4001 3FFF	ADC3		Section 11.12.15 on page 231	0x4000 4400 - 0x4000 47FF	USART2		Section 25.6.8 on page 693
0x4001 3800 - 0x4001 3BFF	USART1		Section 25.6.8 on page 693	0x4000 4000 - 0x4000 3FFF	Reserved	APB1	
0x4001 3400 - 0x4001 37FF	TIM8 timer		Section 13.4.21 on page 317	0x4000 3C00 - 0x4000 3FFF	SPI3/I2S		Section 23.5 on page 614
0x4001 3000 - 0x4001 33FF	SPI1		Section 23.5 on page 614	0x4000 3800 - 0x4000 3BFF	SPI2/I2S		Section 23.5 on page 614
0x4001 2C00 - 0x4001 2FFF	TIM1 timer		Section 13.4.21 on page 317	0x4000 3400 - 0x4000 37FF	Reserved	APB1	
0x4001 2800 - 0x4001 2BFF	ADC2		Section 11.12.15 on page 231	0x4000 3000 - 0x4000 33FF	Independent watchdog (IWDG)		Section 17.4.5 on page 403
0x4001 2400 - 0x4001 27FF	ADC1		Section 11.12.15 on page 231	0x4000 2C00 - 0x4000 2FFF	Window watchdog (WWDG)		Section 18.6.4 on page 408
0x4001 2000 - 0x4001 23FF	GPIO Port G		Section 8.5 on page 167	0x4000 2800 - 0x4000 2BFF	RTC	APB1	Section 16.4.7 on page 398
0x4001 1C00 - 0x4001 1FFF	GPIO Port F		Section 8.5 on page 167	0x4000 1800 - 0x4000 27FF	Reserved		
0x4001 1800 - 0x4001 1BFF	GPIO Port E		Section 8.5 on page 167	0x4000 1400 - 0x4000 17FF	TIM7 timer		Section 15.4.9 on page 386
0x4001 1400 - 0x4001 17FF	GPIO Port D		Section 8.5 on page 167	0x4000 1000 - 0x4000 13FF	TIM6 timer	APB1	Section 15.4.9 on page 386
0x4001 1000 - 0x4001 13FF	GPIO Port C		Section 8.5 on page 167	0x4000 0C00 - 0x4000 0FFF	TIM5 timer		Section 14.4.19 on page 373
0x4001 0C00 - 0x4001 0FFF	GPIO Port B		Section 8.5 on page 167	0x4000 0800 - 0x4000 0BFF	TIM4 timer		Section 14.4.19 on page 373
0x4001 0800 - 0x4001 0BFF	GPIO Port A		Section 8.5 on page 167	0x4000 0400 - 0x4000 07FF	TIM3 timer	APB1	Section 14.4.19 on page 373
0x4001 0400 - 0x4001 07FF	EXTI		Section 9.3.7 on page 181	0x4000 0000 - 0x4000 03FF	TIM2 timer		Section 14.4.19 on page 373
0x4001 0000 - 0x4001 03FF	AFIO		Section 8.5 on page 167				

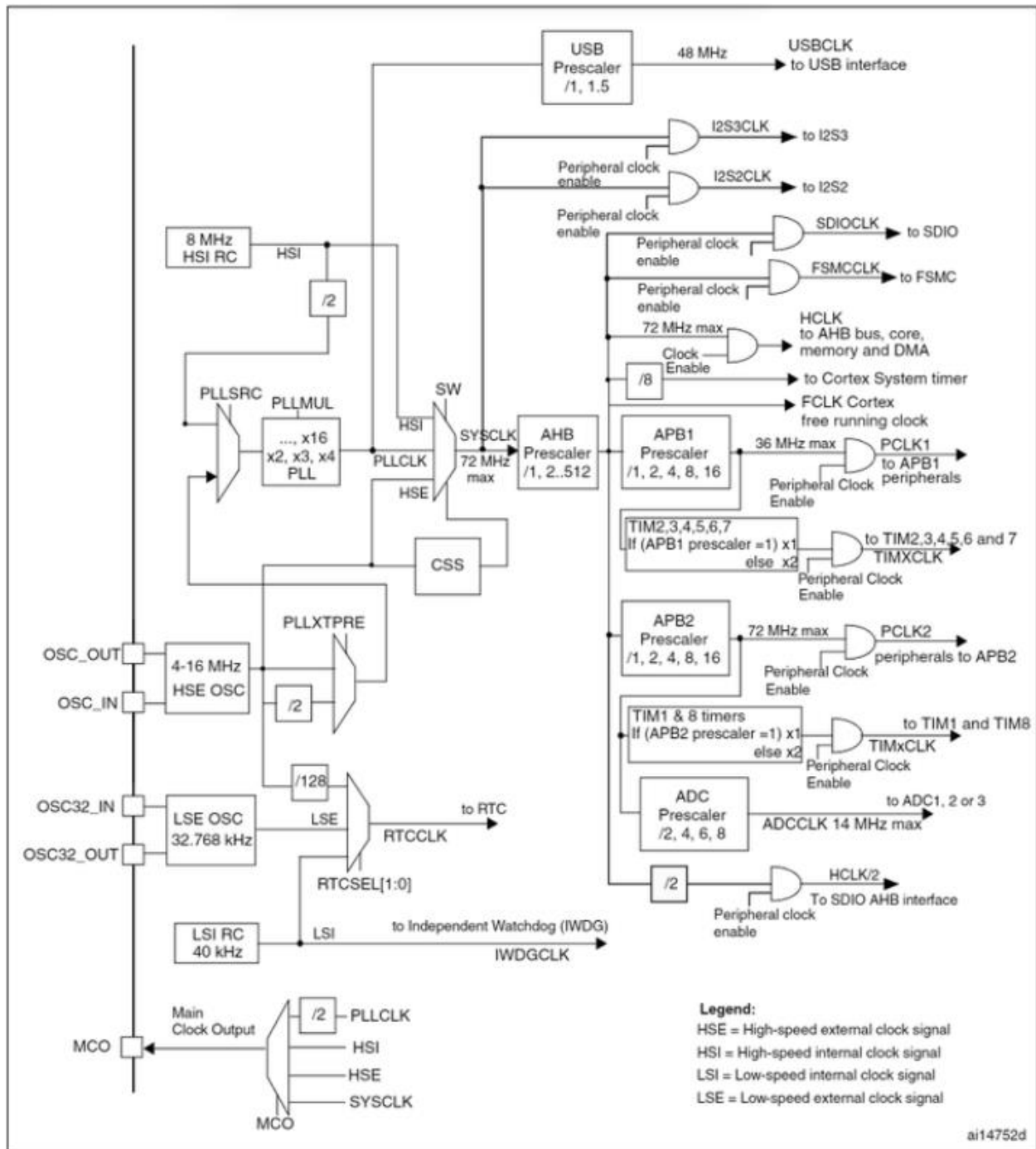
Hình 7: Phạm vi truy cập ngoại vi của bộ vi điều khiển STM32F103C8T6.

- Các ngoại vi của bộ vi điều khiển chỉ được phép truy cập từ địa chỉ 0x40000000 trở đi. Trong đó:
 - o Địa chỉ khởi đầu của AHB là 0x40018000 trùng với địa chỉ khởi đầu của ngoại vi SDIO.
 - o Địa chỉ khởi đầu của APB2 là 0x40010000 trùng với địa chỉ khởi đầu của ngoại vi AFIO.
 - o Địa chỉ khởi đầu của AHB là 0x40000000 trùng với địa chỉ khởi đầu của ngoại vi TIM2.

4. Tìm hiểu về thiết kế cây xung nhịp:

- Trước hết, xung nhịp hệ thống hoạt động dựa trên ba nguồn xung nhịp chính là:
 - o Xung nhịp dao động HSI.
 - o Xung nhịp dao động HSE.

- Xung nhịp lặp khóa pha *PLL*.
- Đồng thời, bộ vi điều khiển cũng được tích hợp sẵn nguồn xung nhịp thứ cấp gồm:
 - Xung nhịp nội mạch RC tốc độ thấp (*LSI RC – 40kHz*) dùng cho bộ đếm thời gian độc lập (*Independent Watchdog – IWDG*) và tùy chọn dùng trên bộ xung nhịp thời gian thực (*Real-time Clock – RTC*) trong trường hợp tự động đánh thức ở chế độ dừng/chờ (*Stop/Standby Mode*).
 - Bộ thạch anh ngoại tốc độ thấp (*LSE External – 32,768kHz*) dùng cho bộ đồng hồ thời gian thực (*Real-time Clock – RTC*).
- Các nguồn xung nhịp này đều có thể được nối thành 1 cây xung nhịp có thể tùy ý tắt/bật để tối ưu điện năng như hình sau:



Hình 8: Cây xung nhịp trên STM32F103C8T6.

4.1. Bộ xung nhịp dao động HSE:

- Tín hiệu có thể được tạo ra từ 2 nguồn xung nhịp:
 - o Bộ cộng hưởng thạch anh/gốm ngoại HSE.

- Nguồn xung nhịp ngoại từ người dùng.
- Bộ cộng hưởng và tụ điện tải phải đặt gần chân dao động để tránh sai lệch đầu ra và thời gian ổn định khởi động.

4.2. Bộ xung nhịp dao động HSI:

- Tín hiệu có thể được tạo ra từ bộ dao động *RC* nội mạch 8MHz và dùng trực tiếp như xung nhịp hệ thống hoặc chia đôi để làm đầu vào bộ xung nhịp lập khóa pha.
- Bộ dao động *HSI RC* cung cấp nguồn xung nhịp ở mức hao mòn thấp với thời gian khởi động nhanh hơn bộ dao động thạch anh ngoại *HSE*, tuy nhiên nếu có hiệu chỉnh thì tần số xung nhịp vẫn kém chính xác hơn bộ *HSE* hoặc bộ cộng hưởng gốm.

4.3. Bộ xung nhịp lập khóa pha PLL:

- Được sử dụng để khuếch đại tần số đầu ra *HSI RC* và *HSE* với hệ số nhân.

4.4. Tùy chọn xung nhịp hệ thống:

- Mặc định thì bộ dao động *HSI* sẽ được lựa chọn để cung cấp xung nhịp.
- Nếu một nguồn xung nhịp đã được lựa chọn thì sẽ không thể ngừng.
- Sự chuyển đổi giữa các nguồn xung nhịp chỉ xảy ra nếu nguồn được lựa chọn đã sẵn sàng (xung nhịp ổn định sau trì hoãn khởi động hoặc bộ *PLL* bị khóa).

4.5. Hệ thống bảo vệ xung nhịp:

- Cho phép kích hoạt bằng chương trình. Trong trường hợp này, khối xác định xung nhịp sẽ được kích hoạt trong trường hợp sau khi bộ dao động *HSE* bị delay và vô hiệu khi bộ dao động bị ngưng.
- Nếu xảy ra lỗi trên xung nhịp dao động *HSE*, bộ dao động sẽ lập tức bị ngắt, tín hiệu thông báo lỗi xung nhịp được truyền đến đầu vào bộ định thời kiểm soát nâng cao (*Timer – TIM1* đến *TIM8*) và một ngắt luồng sẽ được tạo ra để thông báo chương trình về lỗi xung nhịp (*Clock Security System Interrupt – CSSI*), cho phép bộ vi điều khiển (*Micro Controller Unit – MCU*) thực hiện quy trình giải quyết.

- CSSI được liên kết với bộ ngắt luồng vô hiệu hóa chặn (*Non-Maskable Interrupt – NMI*) của bộ vi xử lý.
- Nếu bộ dao động *HSE* được sử dụng trực tiếp cho xung nhịp hệ thống và có xảy ra lỗi trên *HSE* thì xung nhịp hệ thống lập tức chuyển qua *HSI* và vô hiệu hoạt động *HSE*, còn trường hợp xảy ra lỗi khi *HSE* dùng khuếch đại *PLL* thì cả *HSE* và *PLL* đều bị vô hiệu hóa.

5. Cơ chế khởi động chung đối với các ngoại vi:

- Là loại cơ chế ràng buộc giữa ngoại vi và đường dẫn *Bus* đã được quy định cho chính ngoại vi trước khi sử dụng.
- Cơ chế trước hết sẽ hoạt hóa đường dẫn *Bus* của ngoại vi đó thông qua ngoại vi *RCC (Reset and Clock Control)* mà đặc biệt là các thanh ghi con như *RC*, *CFGR*, *AHBENR*, *APB1ENR* và *APB2ENR*.
- Sau đó, chương trình sẽ truy cập tới vị trí ô nhớ đại diện cho ngoại vi trong các thanh ghi này và kích lên mức 1 (*ON*).

6. Các thiết lập đồng bộ chung khi lập trình:

6.1. Từ khóa *volatile*:

- Trong quá trình hoạt động của ngôn ngữ C trên các vi điều khiển, khi chúng ta có nhu cầu đọc giá trị một biến mà tại đó giá trị có thể thay đổi không dự đoán trước nhưng trình biên dịch lúc này lại cố gắng không truy cập vào biến nhiều lần.
- ➔ Lúc này từ khóa *volatile* sẽ cần thiết để ép buộc trình biên dịch đọc lại giá trị của biến đó nhiều lần.
- Để thống nhất đơn giản hóa và tối ưu thời gian lập trình, ta nên định nghĩa nó bằng 1 cụm khác đơn giản hơn nhưng vẫn đảm bảo trình biên dịch và người đánh giá code có thể hiểu:

```
#define vo__ volatile
```

Bảng 1: Thu gọn từ khóa *volatile*.

6.2. Các thiết lập khác:

- Quy định các mức 1 đại diện cho một giá trị được kích hoạt/ thiết lập/... và ngược lại đối với mức 0.

#define	ENABLE	1
#define	DISABLE	0
#define	SET	ENABLE
#define	UNSET	DISABLE
#define	GPIO_PIN_SET	SET
#define	GPIO_PIN_UNSET	UNSET

Bảng 2: Định nghĩa các mức giá trị 0/1.

7. Xử lý thanh ghi và cơ chế lập trình đối với ngoại vi RCC:

7.1. Xử lý thanh ghi:

- Khi làm việc với ngoại vi *RCC* hay bất kỳ ngoại vi khác thông qua thanh ghi thì các đặc điểm quan tâm bao gồm:
 - o Tên thanh ghi.
 - o Độ dài địa chỉ thanh ghi.
 - o Địa chỉ khởi đầu của ngoại vi (trùng với địa chỉ thanh ghi con có độ dài bằng 0).
 - o Giá trị reset.
 - o Ý nghĩa của từng vị trí bit.
 - o Loại truy cập.
- Trong ngoại vi *RCC* tồn tại rất nhiều thanh ghi với nhiều cơ chế khác nhau, tuy nhiên trong nội dung báo cáo chỉ đề cập tới các thanh ghi sau (một số thanh ghi sẽ không được đề cập đầy đủ chi tiết):

7.1.1. Thanh ghi *RCC_CFGR*:

- Dùng để xác định và lựa chọn các thông số trên dây xung nhịp để bộ vi điều khiển hoạt động.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				MCO[3:0]				Res.	OTGF SPRE	PLLMUL[3:0]				PLL XTPRE	PLL SRC
				rW	rW	rW	rW		rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADC PRE[1:0]		PPRE2[2:0]		PPRE1[2:0]			HPRE[3:0]			SWS[1:0]		SW[1:0]			
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	r	r	rW	rW	

Hình 9: Cấu trúc thanh ghi RCC_CFGR.

- Trong đó:
 - Bit 31 đến 28 và bit thứ 23: không sử dụng và được mặc định bằng 0.
 - Bit 27 đến 24: Dùng để set loại xung nhịp đầu ra cho xung nhịp bộ vi điều khiển.
 - Bit 22: Dùng để làm bộ chia tần (*Prescaler*) cho ngoại vi *USB OTG FS*.
 - Bit 21 đến 18: Dùng làm hệ số nhân cho *PLL* (tuy nhiên không được vượt qua 72MHz).
 - Bit 17: Dùng làm hệ số chia của *PREDIV1* dựa trên bit *LSB*.
 - Bit 16: Dùng để xác định đầu vào nguồn cho *PLL*.
 - Bit 15 đến 14: Dùng để làm *Prescaler* cho ngoại vi *ADC*.
 - Bit 13 đến 11: Dùng để làm *Prescaler* cho *APB2*.
 - Bit 10 đến 8: Dùng để làm *Prescaler* cho *APB1*.
 - Bit 7 đến 4: Dùng để làm *Prescaler* cho *AHB*.
 - Bit 3 đến 2: Xác định trạng thái của chuyển đổi xung nhịp hệ thống
 - Bit 1 đến 0: Xác định loại chuyển đổi xung nhịp hệ thống.

7.1.2. Thanh ghi RCC_APB2RSTR:

- Dùng để reset các ngoại vi thuộc *APB2 Bus*.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved																
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Res.	USART1 RST	Res.	SPI1 RST	TIM1 RST	ADC2 RST	ADC1 RST	Reserved			IOPE RST	IOPD RST	IOPC RST	IOPB RST	IOPA RST	Res.	AFIO RST
	rw		rw	rw	rw	rw				rw	rw	rw	rw	rw		rw

Hình 10: Cấu trúc thanh ghi RCC_APB2RSTR.

7.1.3. Thanh ghi RCC_APB1RSTR:

- Dùng để reset các ngoại vi thuộc *APB1 Bus*.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DAC RST	PWR RST	BKP RST	CAN2 RST	CAN1 RST	Reserved		I2C2 RST	I2C1 RST	UART 5 RST	UART 4 RST	USART 3 RST	USART 2 RST	Res.
		rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 RST	SPI2 RST	Reserved		WWD GRST	Reserved					TIM7 RST	TIM6 RST	TIM5 RST	TIM4 RST	TIM3 RST	TIM2 RST
rw	rw			rw						rw	rw	rw	rw	rw	rw

Hình 11: Cấu trúc thanh ghi RCC_APB1RSTR.

7.1.4. Thanh ghi RCC_AHBENR:

- Dùng để kích hoạt xung nhịp cho các ngoại vi thuộc *AHB Bus*.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														ETH MACR XEN	
														rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETHM ACTX EN	ETHM ACEN	Res.	OTGF SEN	Reserved					CRCEN	Res.	FLITFEN	Res.	SRAM EN	DMA2 EN	DMA1 EN
rw	rw		rw						rw		rw		rw	rw	rw

Hình 12: Cấu trúc thanh ghi RCC_AHBENR.

7.1.5. Thanh ghi RCC_APB2ENR:

- Dùng để kích hoạt xung nhịp cho các ngoại vi thuộc *APB2 Bus*.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART1EN	Res.	SPI1 EN	TIM1 EN	ADC2 EN	ADC1 EN	Reserved			IOPE EN	IOPD EN	IOPC EN	IOPB EN	IOPA EN	Res.
	rw		rw	rw	rw	rw				rw	rw	rw	rw	rw	
															AFIO EN
															rw

Hình 13: Cấu trúc thanh ghi RCC_APB2ENR.

7.1.6. Thanh ghi RCC_APB1ENR:

- Dùng để kích hoạt xung nhịp cho các ngoại vi thuộc *APB1 Bus*.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DAC EN	PWR EN	BKP EN	CAN2 EN	CAN1 EN	Reserved		I2C2 EN	I2C1 EN	UART5E N	UART4 EN	USART 3EN	USART 2EN	Res.
		rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWD GEN	Reserved					TIM7 EN	TIM6 EN	TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw						rw	rw	rw	rw	rw	rw

Hình 14: Cấu trúc thanh ghi RCC_APB1ENR.

7.2. Cơ chế lập trình:

- Cơ chế lập trình đối với ngoại vi RCC trong ngôn ngữ C:
 - o Định nghĩa địa chỉ của ngoại vi RCC và biến đại diện cho ngoại vi RCC.

```
#define RCC_BASE_ADDR      0x40021000UL
#define RCC                 ((RCC_OFFSET_TO_REG*)RCC_BASE_ADDR)
```

Bảng 3: Định nghĩa địa chỉ và biến đại diện RCC.

- Cấu trúc hóa bộ thanh ghi cho ngoại vi RCC:

```
typedef struct {
    // vo__ là viết tắt của từ khóa volatile trong ngôn ngữ C.
    //Dùng cho các thiết bị có mật độ thiết kế thấp - trung -
    cao
    vo__ uint32_t _CR;
    //Độ dời địa chỉ: 0x00000000, Giá trị reset: 0x0000XX83
    vo__ uint32_t _CFGR;
    //Độ dời địa chỉ: 0x00000004
    vo__ uint32_t _CIR;
    //Độ dời địa chỉ: 0x00000008
    vo__ uint32_t _APB2RSTR;
    //Độ dời địa chỉ: 0x0000000C
    vo__ uint32_t _ABP1RSTR;
    //Độ dời địa chỉ: 0x00000010
    vo__ uint32_t _AHBENR;
    //Độ dời địa chỉ: 0x00000014, Giá trị reset: 0x00000014
    vo__ uint32_t _APB2ENR;
    //Độ dời địa chỉ: 0x00000018
    vo__ uint32_t _APB1ENR;
```

```

//Độ dài địa chỉ: 0x0000001C
    vo__ uint32_t _BDCR;
//Độ dài địa chỉ: 0x00000020
    vo__ uint32_t _CSR;
//Độ dài địa chỉ: 0x00000024, Giá trị reset: 0x0C000000

    //Dùng cho các thiết bị có đường dẫn kết nối
    vo__ uint32_t _AHBSTR;
//Độ dài địa chỉ: 0x00000028
    vo__ uint32_t _CFGR2;
//Độ dài địa chỉ: 0x0000002C
//Nếu không đề cập tới giá trị reset thì mặc định là 0x00000000
} RCC_OFFSET_TO_REG;

```

Bảng 4: Cấu trúc hóa bộ thanh ghi cho ngoại vi RCC.

- Nếu muốn sử dụng một thanh ghi nào đó trong bộ thanh ghi ngoại vi, ta chỉ cần dùng dấu mũi tên và trở đến thanh ghi mong muốn. Ví dụ:

```

RCC->_APB1ENR
RCC->_APB2ENR
RCC->_RC

```

Bảng 5: Quy tắc sử dụng truy xuất thanh ghi.

- Nếu muốn thiết lập khoảng giá trị với độ dài bit bất kì không vượt quá độ dài thanh ghi thì có 2 cách:
 - Có thể quy đổi khoảng bit đó thành hệ thập phân và dịch trái (do cơ chế thanh ghi khi biểu diễn sẽ bắt đầu từ 31 về 0).
 - Thiết lập mức 1/ mức 0 rồi dịch trái / phải bao nhiêu đơn vị để truy cập tới ô nhớ bit mong muốn.
 - Sau khi đã truy cập tới ô nhớ mong muốn thì ta phải dùng toán tử OR (ký hiệu “|”) để OR chính nó và cập nhật thanh ghi.

```

RCC->_APB1ENR |= (1 << 0)

```

Bảng 6: Quy tắc set giá trị cho ô bit trên thanh ghi.

8. Giải mã mạch và cơ chế hoạt động của ngoại vi GPIO:

8.1. Giải mã mạch:

- Trong mỗi cổng *GPIO* có 2 thanh ghi thiết lập loại 32-bit (*GPIOx_CRL*, *GPIOx_CRH*), 2 thanh ghi dữ liệu 32-bit (*GPIOx_IDR*, *GPIOx_ODR*), 1 thanh ghi set/reset 32-bit (*GPIOx_BSRR*), 1 thanh ghi reset 16-bit (*GPIOx_BRR*) và 1 thanh ghi khóa 32-bit (*GPIOx_LCKR*) được tự do lập trình với truy cập duy nhất 32-bit.
- Dựa trên đặc điểm phân cứng của mỗi cổng *GPIO* mà có thể thiết lập theo các dạng sau:
 - o Đầu vào không xác định mức (*Input floating*).
 - o Đầu vào kéo lên (*Input pull-up*).
 - o Đầu vào kéo xuống (*Input pull-down*).
 - o Đầu vào tín hiệu Analog (*Analog Input*).
 - o Đầu ra xả mở (*Output Open-Drain*).
 - o Đầu ra kéo thả (*Output Push-Pull*).
 - o Chức năng thay thế dạng kéo thả (*Alternate function Push-Pull*).
 - o Chức năng thay thế dạng mở xả (*Alternate function Open-Drain*).
- Tuy nhiên để cho phép lựa chọn chuyển đổi thì ta sẽ dựa vào bảng cấu hình thiết lập và đầu ra:

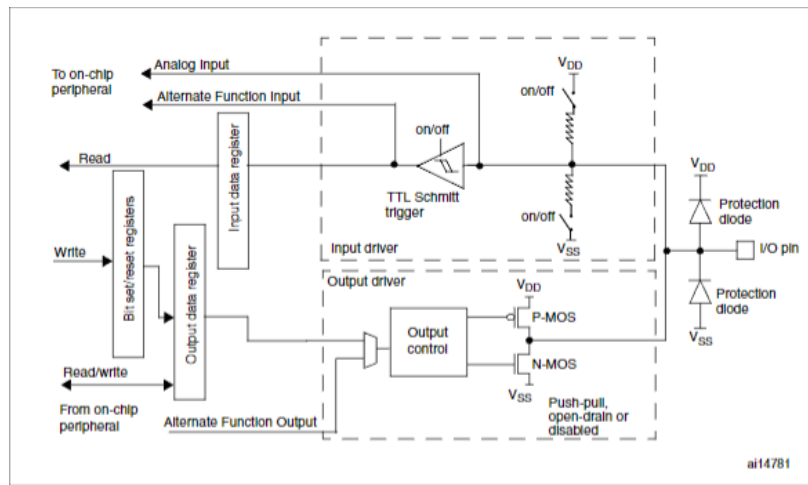
Cấu hình thiết lập		CNF1	CNF0	MODE 1	MODE 0	PxODR		
Đầu ra	Push-Pull	0	0	Tra bảng bên dưới		0/1		
	Open-Drain		1			0/1		
Chức năng thay thế	Push-Pull	1	0			N/A		
	Open-Drain		1			N/A		
Đầu vào	Tín hiệu Analog	0	0	00		N/A		
	Floating (mặc định)		1			N/A		
	Pull-up	1	0			0		
	Pul-down					1		

Bảng 7: Bảng cấu hình bit CNF, bit MODE và ODRx.

MODE [1:0]	Ý nghĩa
00	Chế độ đầu vào
01	Đầu ra tốc độ tối đa 10MHz
10	Đầu ra tốc độ tối đa 2MHz
11	Đầu ra tốc độ tối đa 50MHz

Bảng 8: Bảng cấu hình bit MODE chuyên sâu.

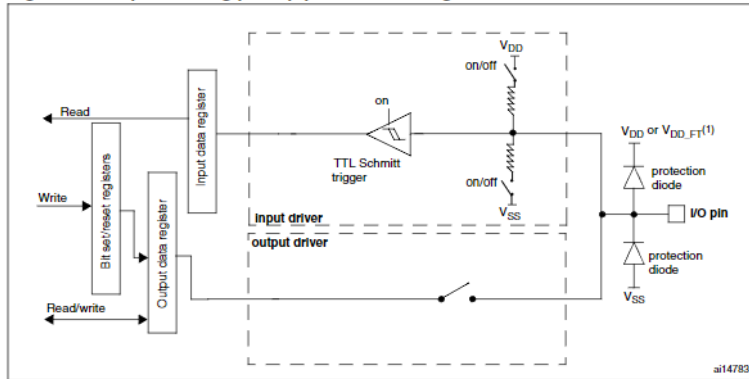
- Cấu trúc cơ bản của mỗi chân *GPIO*:
 - o 2 khối điều khiển đầu vào đầu ra.
 - o 2 *diode* bảo vệ mạch.



Hình 15: Cấu trúc cơ bản chân *GPIO*.

8.2. Cơ chế hoạt động nếu được thiết lập là đầu vào:

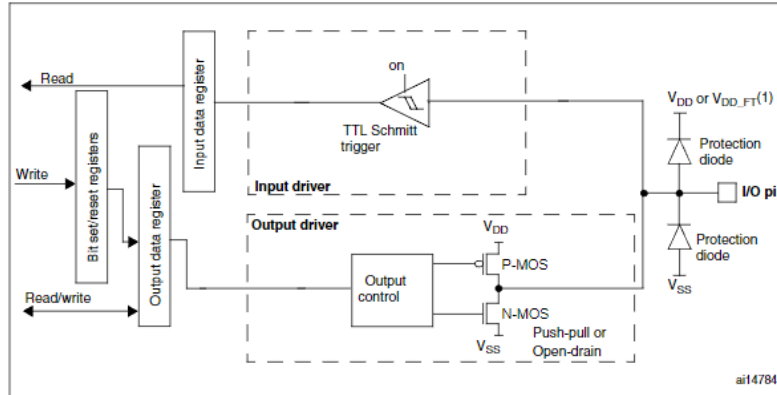
- Bộ đệm đầu ra bị vô hiệu.
- Bộ kích hoạt đầu vào *Schmitt* được kích hoạt.
- Các bộ trở nội kéo lên/xuống được kích hoạt hay vô hiệu tùy thuộc vào cấu hình thiết lập (*pull-up*, *pull-down*, không xác định – *floating*).
- Gói dữ liệu hiện diện trong chân *I/O* sẽ được nạp vào thanh ghi `GPIOx_IDR` mỗi chu kỳ xung nhịp của *APB2 Bus*.
- Các truy cập đọc tới thanh ghi `GPIOx_IDR` sẽ tích hợp trạng thái chân *I/O*.



Hình 16: Chân GPIO đầu vào.

8.3. Cơ chế hoạt động nếu được thiết lập là đầu ra:

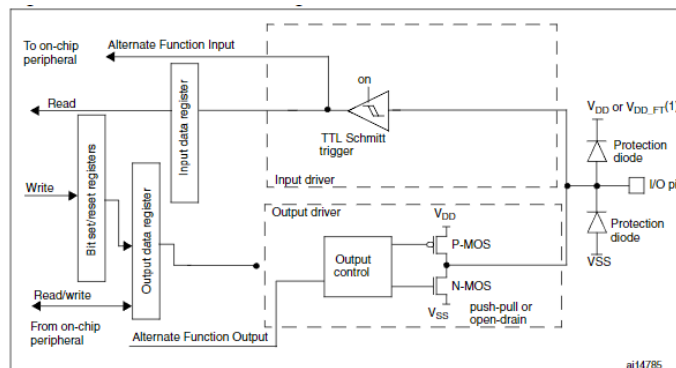
- Bộ đệm đầu ra sẽ được kích hoạt:
 - Chế độ *Open-Drain*: mức 0 trong thanh ghi `GPIOx_ODR` kích hoạt khối *N-MOS* trong khi mức 1 sẽ làm đầu ra ở trạng thái không cố định điện áp (trạng thái *Hi-Z – High Impedence*) và khối *P-MOS* không bao giờ được kích hoạt.
 - Chế độ *Push-Pull*: mức 0 trong thanh ghi `GPIOx_ODR` kích hoạt khối *N-MOS* (đầu ra *N-MOS* sẽ là 1) trong khi mức 1 sẽ kích hoạt khối *P-MOS* (đầu ra *P-MOS* sẽ là 0).
- Bộ kích hoạt đầu vào *Schmitt* được kích hoạt.
- Các bộ trở nội kéo lên/xuống được vô hiệu.
- Gói dữ liệu hiện diện trong chân I/O sẽ được nạp vào thanh ghi `GPIOx_IDR` mỗi chu kỳ xung nhịp của *APB2 Bus*.
- Các truy cập đọc tới thanh ghi `GPIOx_IDR` sẽ trả về trạng thái chân I/O ở chế độ *Open-Drain*.
- Các truy cập đọc tới thanh ghi `GPIOx_ODR` sẽ trả về giá trị cuối cùng được ghi ở chế độ *Push-Pull*.



Hình 17: Chân GPIO đầu ra.

8.4. Cơ chế hoạt động nếu được thiết lập là chức năng thay thế:

- Bộ đệm đầu ra sẽ được kích hoạt ở chế độ *Open-Drain* hoặc *Push-Pull*.
- Bộ đệm đầu ra hoạt động bởi tín hiệu đến từ ngoại vi được chọn thay thế chức năng cho *GPIO*.
- Bộ kích hoạt đầu vào *Schmitt* được kích hoạt.
- Các bộ trở nội kéo lên/xuống được vô hiệu.
- Gói dữ liệu hiện diện trong chân *I/O* sẽ được nạp vào thanh ghi `GPIOx_IDR` mỗi chu kỳ xung nhịp của *APB2 Bus*.
- Các truy cập đọc tới thanh ghi `GPIOx_IDR` sẽ trả về trạng thái chân *I/O* ở chế độ *Open-Drain*.
- Các truy cập đọc tới thanh ghi `GPIOx_ODR` sẽ trả về giá trị cuối cùng được ghi ở chế độ *Push-Pull*.



Hình 18: Chân GPIO tín hiệu Analog.

9. Xử lý thanh ghi và cơ chế lập trình đối với ngoại vi GPIO:

9.1. Xử lý thanh ghi:

- Khi có nhu cầu giao tiếp với các ngoại vi (ở đây ta sẽ giả sử trong trường hợp này là *GPIO*), ta cần phải quan tâm đến các thông tin sau:
 - o Tên cổng *GPIO* (từ A tới G).
 - o Thứ tự của cổng (từ 1 tới 15).
 - o Xác định địa chỉ khởi đầu của cổng *GPIO* (từ A đến G).
 - o Xác định địa chỉ khởi đầu của đường dẫn truyền dữ liệu mà cổng *GPIO* đó được quy định (có thể là đường dẫn *AHB*, *APB1* và *APB2*).
 - o Xác định trạng thái hoạt động mong muốn.
- Trong ngoại vi *GPIOx* (từ A tới G) tồn tại các thanh ghi với nhiều cơ chế - công dụng khác nhau, tuy nhiên trong nội dung báo cáo chỉ đề cập tới các thanh ghi sau (một số thanh ghi sẽ không được đề cập đầy đủ chi tiết):

9.1.1. Thanh ghi *GPIOx_CRL*:

- Dùng để cài đặt *CNF* và *MODE* cho các chân có thứ tự từ 0 tới 7.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Hình 19: Cấu trúc thanh ghi *GPIOx_CRL*.

- Bảng tùy chỉnh loại *CNF* và *MODE* nằm ở mục 8.1.
- Để truy cập tới vị trí *MODE* bất kỳ thì ta sẽ dùng công thức:

$$\text{Giá trị điền vào} = \text{MODE của pin} \ll (4 \times \text{Số thứ tự của pin})$$

Bảng 9: Công thức truy cập bit *MODE*.

- Để truy cập tới vị trí *CNF* bất kỳ thì ta sẽ dùng công thức:

$$\text{Giá trị điền vào} = \text{CNF của pin} \ll (4 \times \text{Số thứ tự của pin} + 2)$$

Bảng 10: Công thức truy cập bit *CNF*.

9.1.2. Thanh ghi GPIOx_CRH:

- Dùng để cài đặt CNF và MODE cho các chân có thứ tự từ 8 tới 15.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Hình 20: Cấu trúc thanh ghi GPIOx_CRH.

- Bảng tùy chỉnh loại CNF và MODE nằm ở mục 8.1.
- Do thứ tự bit tính từ 0 nên phải có bước chuyển đổi về thứ tự 0 tới 7 thông qua công thức.

$$\text{Thứ tự pin chuyển đổi} = \text{thứ tự ban đầu của pin} \% 8$$

Bảng 11: Công thức chuyển đổi pin.

- Các công thức truy cập tương tự với thanh ghi GPIOx_CRL.

9.1.3. Thanh ghi GPIOx_IDR:

- Dùng để đọc giá trị truyền từ đầu vào.
- Chỉ cho phép quyền đọc và truy cập ở mức Word.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Hình 21: Cấu trúc thanh ghi GPIOx_IDR.

9.1.4. Thanh ghi GPIOx_ODR:

- Dùng để đọc giá trị truyền qua đầu ra.
- Cho phép đọc và ghi bằng chương trình.
- Để truy cập tới cấp độ bit, ta nên dùng thanh ghi GPIOx_BSRR.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Hình 22: Cấu trúc thanh ghi GPIOx_ODR.

9.1.5. Thanh ghi GPIOx_BSRR:

- Dùng để set/reset giá trị cho các chân.
- Chỉ cho phép ghi giá trị và truy cập ở mức *Word*.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Hình 23: Cấu trúc thanh ghi GPIOx_BSRR.

- Trong đó:
 - BR tương ứng với reset chân.
 - BS tương ứng với set chân.

9.2. Cơ chế lập trình:

- Định nghĩa địa chỉ của ngoại vi *GPIOx* (x từ A tới G) và biến đại diện cho ngoại vi *GPIOx*:

```
#define GPIOA_BASE_ADDR    0x40010800UL
#define GPIOB_BASE_ADDR    0x40010C00UL
#define GPIOC_BASE_ADDR    0x40011000UL
#define GPIOD_BASE_ADDR    0x40011400UL
#define GPIOE_BASE_ADDR    0x40011800UL
#define GPIOF_BASE_ADDR    0x40011C00UL
#define GPIOG_BASE_ADDR    0x40012000UL
#define GPIOA              ((GPIO_OFFSET_TO_REG*)GPIOA_BASE_ADDR)
#define GPIOB              ((GPIO_OFFSET_TO_REG*)GPIOB_BASE_ADDR)
#define GPIOC              ((GPIO_OFFSET_TO_REG*)GPIOC_BASE_ADDR)
#define GPIOD              ((GPIO_OFFSET_TO_REG*)GPIOD_BASE_ADDR)
#define GPIOE              ((GPIO_OFFSET_TO_REG*)GPIOE_BASE_ADDR)
#define GPIOF              ((GPIO_OFFSET_TO_REG*)GPIOF_BASE_ADDR)
#define GPIOG              ((GPIO_OFFSET_TO_REG*)GPIOG_BASE_ADDR)
```

Bảng 12: Định nghĩa địa chỉ và biến đại diện cho GPIO.

- Cấu trúc hóa bộ thanh ghi cho *GPIOx*:

```

typedef struct {
    //Dùng cho các cổng GPIO từ A tới G
    vo__ uint32_t _CRL;
    //Độ dài địa chỉ: 0x00000000, Giá trị reset: 0x44444444
    vo__ uint32_t _CRH;
    //Độ dài địa chỉ: 0x00000004, Giá trị reset: 0x44444444
    vo__ uint32_t _IDR;
    //Độ dài địa chỉ: 0x00000008, Giá trị reset: 0x0000XXXX (X không xác định)
    vo__ uint32_t _ODR;
    //Độ dài địa chỉ: 0x0000000C
    vo__ uint32_t _BSRR;
    //Độ dài địa chỉ: 0x00000010
    vo__ uint32_t _BRR;
    //Độ dài địa chỉ: 0x00000014
    vo__ uint32_t _LCKR;
    //Độ dài địa chỉ: 0x00000018
    //Giá trị reset: 0x00000000 (Nếu không đề cập tới giá trị reset)
} GPIO_OFFSET_TO_REG;

```

Bảng 13: Cấu trúc bộ thanh ghi cho GPIO.

- Định nghĩa macro cho khởi động và vô hiệu xung clock cho *GPIO* bằng *RCC*:

```

#define GPIOA_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 2)
#define GPIOB_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 3)
#define GPIOC_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 4)
#define GPIOD_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 5)
#define GPIOE_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 6)
#define GPIOF_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 7)
#define GPIOG_PERI_CLK_ENB()    RCC->_APB2ENR |= (1 << 8)
/*-----*/
#define GPIOA_PERI_CLK_DENB()    RCC->_APB2ENR &= ~(1 << 2)
#define GPIOB_PERI_CLK_DENB()    RCC->_APB2ENR &= ~(1 << 3)
#define GPIOD_PERI_CLK_DENB()    RCC->_APB2ENR &= ~(1 << 5)
#define GPIOE_PERI_CLK_DENB()    RCC->_APB2ENR &= ~(1 << 6)
#define GPIOF_PERI_CLK_DENB()    RCC->_APB2ENR &= ~(1 << 7)
#define GPIOG_PERI_CLK_DENB()    RCC->_APB2ENR &= ~(1 << 8)

```

Bảng 14: Định nghĩa macro cho khởi động và vô hiệu xung clock cho GPIO.

- Trong đó, để ngắt kích hoạt xung clock, ta dùng toán tử AND “&”. Khi này, nếu thiết lập mức 1 và dịch bit tới vị trí muốn xóa thì ta sẽ thêm toán tử NOT “~” trước khối dịch mức để biến thành mức 0 và thực hiện AND.
- Cấu trúc hóa các thành phần sử dụng trong tương tác với *GPIO*:

```
typedef struct {
    uint8_t __PIN_NUMBER;
    uint8_t __PIN_MODE;
    uint8_t __PIN_CFG_TYPE;
    bool __PIN_PU_PD;
    bool __PIN_RE_T;
    bool __PIN_FE_T;
} GPIO_PIN_CFG;

typedef struct {
    GPIO_OFFSET_TO_REG *pGPIOx;
    GPIO_PIN_CFG GPIO_Pin_Config;
} GPIO_HANDLER;
```

Bảng 15: Cấu trúc hóa các thành phần sử dụng trong tương tác với *GPIO*.

- *GPIO_PIN_CFG* có tác dụng lưu trữ số thứ tự của chân, chế độ hoạt động và loại cấu hình.
- *GPIO_HANDLER* có tác dụng lưu cổng *GPIO* và bộ thanh ghi đã được cấu trúc hóa.

10. Xử lý tạo lập trình điều khiển ngoại vi *GPIO*:

- Để khai thác tối đa hiệu quả sử dụng ngoại vi *GPIO*, ta cũng cần định nghĩa thêm các hàm hỗ trợ như sau:
 - Hàm *GPIO_Clock_Control*:
 - Dùng để khởi động/vô hiệu xung clock cho cổng *GPIO*.

```
void GPIO_Clock_Control(GPIO_OFFSET_TO_REG *pGPIOX, uint8_t ENorDI){
    if (ENorDI == ENABLE) {
        if (pGPIOX == GPIOA)
            GPIOA_PERI_CLK_ENB();
    }
}
```



```

        else if (pGPIOX == GPIOB)
            GPIOB_PERI_CLK_ENB();
        else if (pGPIOX == GPIOC)
            GPIOC_PERI_CLK_ENB();
        else if (pGPIOX == GPIOD)
            GPIOD_PERI_CLK_ENB();
        else if (pGPIOX == GPIOE)
            GPIOE_PERI_CLK_ENB();
        else if (pGPIOX == GPIOF)
            GPIOF_PERI_CLK_ENB();
        else if (pGPIOX == GPIOG)
            GPIOG_PERI_CLK_ENB();
    } else {
        if (pGPIOX == GPIOA)
            GPIOA_PERI_CLK_DENB();
        else if (pGPIOX == GPIOB)
            GPIOB_PERI_CLK_DENB();
        else if (pGPIOX == GPIOC)
            GPIOC_PERI_CLK_DENB();
        else if (pGPIOX == GPIOD)
            GPIOD_PERI_CLK_DENB();
        else if (pGPIOX == GPIOE)
            GPIOE_PERI_CLK_DENB();
        else if (pGPIOX == GPIOF)
            GPIOF_PERI_CLK_DENB();
        else if (pGPIOX == GPIOG)
            GPIOG_PERI_CLK_DENB();
    }
}

```

Bảng 16: đoạn mã GPIO_Clock_Control.

○ Hàm GPIO_Init:

- Dùng để khởi tạo giá trị và các thông số cho cổng *GPIO*.

```

void GPIO_Init(GPIO_HANDLER *pGPIO_Handler){
    pGPIO_Handler->pGPIOx->_BRR  &= 0x00000000;
    pGPIO_Handler->pGPIOx->_BSRR &= 0x00000000;
}

```

```

pGPIO_Handler->pGPIOx->_CRH  &= 0x00000000;
pGPIO_Handler->pGPIOx->_CRL  &= 0x00000000;
pGPIO_Handler->pGPIOx->_IDR  &= 0x00000000;
pGPIO_Handler->pGPIOx->_LCKR &= 0x00000000;
pGPIO_Handler->pGPIOx->_ODR  &= 0x00000000;

uint32_t fill_in_value      = UNSET;
uint8_t pin_number          = pGPIO_Handler->GPIO_Pin_Config.__PIN_NUMBER;
uint8_t pin_mode            = pGPIO_Handler->GPIO_Pin_Config.__PIN_MODE;
uint8_t pin_cnf             = pGPIO_Handler->GPIO_Pin_Config.__PIN_CFG_TYPE;
int pin_pull_up_down        = pGPIO_Handler->GPIO_Pin_Config.__PIN_PU_PD;

//Set giá trị cho CNF
if (0 <= pin_number && pin_number <= 7) {
    //This condition is used for configure __PIN_NUMBER from 0 to 7
    fill_in_value = (pin_cnf << (4 * pin_number + 2));
    pGPIO_Handler->pGPIOx->_CRL &= ~(0x3 << (4 * pin_number + 2));
    pGPIO_Handler->pGPIOx->_CRL |= fill_in_value;
} else {
    //This condition is used for configure __PIN_NUMBER from 8 to 15
    uint8_t highpin_convert = pin_number % 8;
    fill_in_value = (pin_cnf << (4 * highpin_convert + 2));
    pGPIO_Handler->pGPIOx->_CRH &= ~(0x3 << (4 * highpin_convert + 2));
    pGPIO_Handler->pGPIOx->_CRH |= fill_in_value;
}

fill_in_value -= fill_in_value;

//Set giá trị cho MODE
if (0 <= pin_number && pin_number <= 7) {
    //Pin từ 1 tới 7
    fill_in_value = (pin_mode << (4 * pin_number));
    pGPIO_Handler->pGPIOx->_CRL &= ~(0x3 << (4 * pin_number));
    pGPIO_Handler->pGPIOx->_CRL |= fill_in_value;
} else {
    //Pin từ 8 tới 15
    uint8_t highpin_convert = pin_number % 8;
    fill_in_value = (pin_mode << (4 * highpin_convert));

```

```

        pGPIO_Handler->pGPIOx->_CRH &= ~(0x3 << (4 * highpin_convert));
        pGPIO_Handler->pGPIOx->_CRH |= fill_in_value;
    }

    fill_in_value -= fill_in_value;

    //Set loại Pull-up / Pull-down
    if (pin_mode == 0) {
        if (pGPIO_Handler->GPIO_Pin_Config.__PIN_PU_PD == true) {
            fill_in_value = (pin_pull_up_down << pin_number);
            pGPIO_Handler->pGPIOx->_ODR |= fill_in_value;
        }
    } else {
        fill_in_value -= fill_in_value;
    }
}

```

Bảng 17: đoạn mã GPIO_Init.

- Hàm GPIO_De_Init:
 - Dùng để xóa các thông số thiết lập cho *GPIO* bằng cách reset xung nhịp.

```

void GPIO_De_Init(GPIO_OFFSET_TO_REG *pGPIOX) {
    if (pGPIOX == GPIOA)
        GPIOA_PERI_CLK_RESET();
    else if (pGPIOX == GPIOB)
        GPIOB_PERI_CLK_RESET();
    else if (pGPIOX == GPIOC)
        GPIOC_PERI_CLK_RESET();
    else if (pGPIOX == GPIOD)
        GPIOD_PERI_CLK_RESET();
    else if (pGPIOX == GPIOE)
        GPIOE_PERI_CLK_RESET();
    else if (pGPIOX == GPIOF)
        GPIOF_PERI_CLK_RESET();
    else if (pGPIOX == GPIOG)

```

```

GPIOG_PERI_CLK_RESET();
}

```

Bảng 18: đoạn mã GPIO_De_Init.

- Trong đó các macro GPIOx_PERI_CLK_RESET() được định nghĩa có dạng như sau:

```

#define GPIOA_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 2);RCC->_APB2RSTR
&= ~(1 << 2);} while (0)
#define GPIOB_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 3);RCC->_APB2RSTR
&= ~(1 << 3);} while (0)
#define GPIOC_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 4);RCC->_APB2RSTR
&= ~(1 << 4);} while (0)
#define GPIOD_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 5);RCC->_APB2RSTR
&= ~(1 << 5);} while (0)
#define GPIOE_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 7);RCC->_APB2RSTR
&= ~(1 << 7);} while (0)
#define GPIOF_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 8);RCC->_APB2RSTR
&= ~(1 << 8);} while (0)
#define GPIOG_PERI_CLK_RESET() do {RCC->_APB2RSTR |= (1 << 9);RCC->_APB2RSTR
&= ~(1 << 9);} while (0)

```

Bảng 19: đoạn mã GPIOx_PERI_CLK_RESET().

- Hàm GPIO_Read_From_Pin:

- Dùng để đọc giá trị (1 hoặc 0) và trả về từ một chân đơn lẻ.

```

uint8_t GPIO_Read_From_Pin(GPIO_OFFSET_TO_REG *pGPIOX, uint8_t PinNumber) {
    uint8_t value;
    value = (uint8_t)((pGPIOX->_IDR >> PinNumber) & 0x00000001);
    return value;
}

```

Bảng 20: đoạn mã GPIO_Read_From_Pin.

- Trong đó, việc phải dùng toán tử AND và dịch phải với 0x00000001 cho phép đọc giá trị của một ô duy nhất mà ta cần.

- Hàm GPIO_Read_From_Port:

- Dùng để đọc giá trị (1 hoặc 0) và trả về từ một cổng.

```
uint16_t GPIO_Read_From_Port(GPIO_OFFSET_TO_REG *pGPIOX) {
    uint16_t value;
    value = (uint16_t)(pGPIOX->_IDR);
    return value;
}
```

Bảng 21: đoạn mã GPIO_Read_From_Port.

- Trong đó, không phải dùng toán tử AND và dịch phải vì trả về cả 1 cổng.
- Hàm GPIO_Write_To_Pin:
 - Dùng để ghi giá trị (1 hoặc 0) cho một chân đơn lẻ.

```
void GPIO_Write_To_Pin(GPIO_OFFSET_TO_REG *pGPIOX, uint8_t PinNumber, uint32_t Value) {
    pGPIOX->_ODR &= 0x00000000;
    pGPIOX->_IDR &= 0x00000000;
    pGPIOX->_BSRR &= 0x00000000;
    if (Value == GPIO_PIN_SET) {
        pGPIOX->_BSRR = (0x1 << (PinNumber + 16));
    } else {
        pGPIOX->_BSRR = (0x1 << PinNumber);
    }
}
```

Bảng 22: đoạn mã GPIO_Write_To_Pin.

- Hàm GPIO_Write_To_Port:
 - Dùng để ghi giá trị cho nguyên một cổng.

```
void GPIO_Write_To_Port(GPIO_OFFSET_TO_REG *pGPIOX, uint16_t Value) {
    pGPIOX->_ODR = Value;
}
```

Bảng 23: đoạn mã GPIO_Write_To_Port.

- Hàm GPIO_Toggle_Pin:
 - Dùng để bật tắt chân trong một lần.

```
void GPIO_Toggle_Pin(GPIO_OFFSET_TO_REG *pGPIOX, uint8_t PinNumber) {
```

```

pGPIOX->_ODR ^= (1 << PinNumber);
}

```

Bảng 24: đoạn mã GPIO_Toggle_Pin.

11. Làm việc với bảng mã Vector:

- Bảng mã Vector (*bảng mã ngoại lệ*) là loại bảng chứa giá trị reset của con trỏ stack và địa chỉ khởi đầu cho các xử lý ngoại lệ như tóm tắt hình sau:

Exception number	IRQ number	Offset	Vector
83	67	0x014C	IRQ67
.	.	.	.
.	.	.	.
18	2	0x004C	IRQ2
17	1	0x0048	IRQ1
16	0	0x0044	IRQ0
15	-1	0x0040	Systick
14	-2	0x003C	PendSV
13		0x0038	Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			Reserved
9			
8			
7			
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	NMI
1		0x0004	Reset
		0x0000	Initial SP value

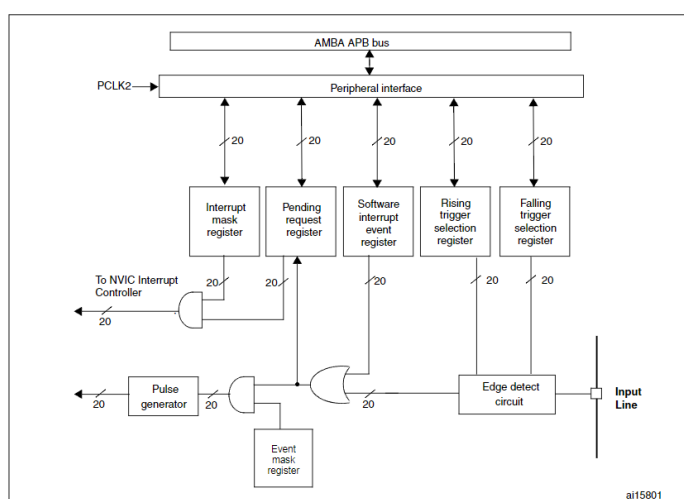
Hình 24: Bảng Vector tóm tắt.

- Trong đó, ta có thể thấy:
 - o *Exception number* – chỉ số IRQ là đại diện cho vị trí của 1 luồng xử lý ngoại lệ trong NVIC.
 - o *IRQ number* – mức ưu tiên là đại diện cho việc bộ xử lý sẽ ưu tiên xử lý luồng nào hơn khi đa yêu cầu về xử lý ngoại lệ đổ về hệ thống. Số IRQ có thể được lập trình bởi chương trình.

- Địa chỉ khởi đầu của bảng vector cố định trong hệ thống là $0x00000000$. Tuy nhiên, các chương trình được cấp quyền có thể viết lại địa chỉ của bảng từ khoảng $0x00000080$ đến $0x3FFFFFFF80$.
- Tương ứng với từng số *IRQ* là một xử lý ngoại lệ cho hệ thống và các ngoại vi phụ trợ.

12. Tìm hiểu về thiết kế ngắt luồng ngoài:

- Đối với bộ vi điều khiển *STM32F103C8T6*, ta được nhà sản xuất cung cấp thiết kế như sau:



Hình 25: Sơ đồ ngắt luồng ngoài - EXTI.

- Dựa vào sơ đồ thiết kế, để cho ngắt luồng hoạt động thì:
 - Luồng đầu vào (tức *Input Line*) có thể xuất phát từ một ngoại vi thuộc bộ vi điều khiển hoặc từ chân *I/O*.
 - Phía đầu vào từ giao diện giao tiếp ngoại vi phải trả về dữ liệu cho thanh ghi xác định nhịp chuyển đổi cạnh lên/xuống (*EXTI_FTSR/EXTI_RTSR*), thanh ghi nhận diện sự kiện ngắt luồng phía chương trình (*EXTI_SWIER*), thanh ghi nạp chờ yêu cầu ngắt luồng (*EXTI_PR*) và thanh ghi phủ ngắt luồng (*EXTI_IMR*) và ngược lại các thanh ghi cũng trả về dữ liệu cho phía ngoại vi.

- Dữ liệu từ thanh ghi `EXTI_SWIER` sẽ kết hợp với mạch xác định cạnh tại cổng OR và truyền tín hiệu tới cổng AND chờ dữ liệu của thanh ghi phủ sự kiện hệ thống (`EXTI_EMR`) và truyền tới để tạo ra xung.
- Dữ liệu từ 2 thanh ghi `EXTI_EMR` và `EXTI_PR` sẽ được nạp qua cổng AND và gửi tới bộ điều khiển ngắt chồng luồng Vector (gọi tắt là *NVIC - Nested vectored interrupt controller*) để xác định và tạo ra luồng ngắt riêng.

13. Xử lý thanh ghi ngắt luồng ngoài trên GPIO:

- Khi có nhu cầu truy cập tới ngắt luồng ngoài (*EXTI - External interrupt/event controller*) đối với, ta cần quan tâm tới các thông tin:
 - Cổng *GPIO*.
 - Số thứ tự chân *GPIO*.
 - Nhịp cạnh xảy ra ngắt luồng.
- Trong *EXTI* tồn tại các thanh ghi với nhiều cơ chế - công dụng khác nhau, tuy nhiên trong nội dung báo cáo chỉ đề cập tới các thanh ghi sau (một số thanh ghi sẽ không được đề cập đầy đủ chi tiết):

13.1. Thanh ghi EXTI_IMR:

- Dừng để phủ/ xóa phủ đối với line ngắt luồng.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												MR19	MR18	MR17	MR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 26: Cấu trúc thanh ghi EXTI_IMR.

13.2. Thanh ghi EXTI_EMR:

- Dừng để phủ/ xóa phủ đối với line ngắt luồng khi có sự kiện hệ thống.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												MR19	MR18	MR17	MR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 27: Cấu trúc thanh ghi EXTI_EMR.

13.3. Thanh ghi EXTI_RSTR:

- Dừng để thiết lập ghi nhận các lần nhíp nhảy cạnh từ mức thấp lên cao.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												TR19	TR18	TR17	TR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 28: Cấu trúc thanh ghi EXTI_RTSR.

13.4. Thanh ghi EXTI_FTSR:

- Dừng để thiết lập ghi nhận các lần nhíp nhảy cạnh từ mức cao xuống thấp.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												TR19	TR18	TR17	TR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 29: Cấu trúc thanh ghi EXTI_FTSR.

13.5. Thanh ghi EXTI_SWIER:

- Dùng để thiết lập ghi nhận các thay đổi trong chương trình khi chạy.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												SWIER 19	SWIER 18	SWIER 17	SWIER 16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER 15	SWIER 14	SWIER 13	SWIER 12	SWIER 11	SWIER 10	SWIER 9	SWIER 8	SWIER 7	SWIER 6	SWIER 5	SWIER 4	SWIER 3	SWIER 2	SWIER 1	SWIER 0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Hình 30: Cấu trúc thanh ghi EXTI_SWIER.

13.6. Thanh ghi EXTI_PR:

- Dùng để ghi nhận trạng thái nạp chờ luồng ngắt khi xảy ra chuyển đổi nhịp cạnh hoặc thay đổi trong hệ thống.
- Chỉ cho phép ghi và xóa đồng thời thông qua mức 1, các thao tác với mức 0 không có tác dụng.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved												PR19	PR18	PR17	PR16
												rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Hình 31: Cấu trúc thanh ghi EXTI_PR.

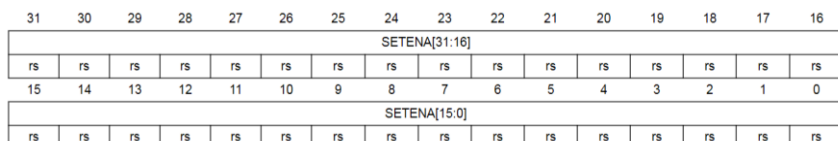
14. Xử lý thanh ghi bộ điều khiển ngắt chồng luồng Vector:

- Khi có nhu cầu thao tác ngắt luồng, ngoài việc phải xử lý cơ chế ở phía ngoại vi (sẽ được đề cập ở các phần sau), ta còn phải quan tâm đến phía bộ vi xử lý mà đại diện là bộ điều khiển ngắt chồng luồng Vector (NVIC), trong đó các thông tin cần chú ý là:
 - Số IRQ của đường dẫn ngắt luồng ngoài (thường trong bảng sẽ có tên là EXTI0 tới 4, EXTI9-5, EXTI10-15,...)
 - Vị trí thiết lập trên thanh ghi NVIC_ISER.
 - Số thứ tự của thanh ghi chứa khối thiết lập của số IRQ trong nhóm thanh ghi NVIC_IPR.

- Trong *NVIC* tồn tại các thanh ghi với nhiều cơ chế - công dụng khác nhau, tuy nhiên trong nội dung báo cáo chỉ đề cập tới các thanh ghi sau (một số thanh ghi sẽ không được đề cập đầy đủ chi tiết do giới hạn về số *IRQ*):

14.1. Thanh ghi *NVIC_ISERx* (x từ 0 tới 2):

- Dùng để thiết lập cho phép ngắt luồng dựa theo số *IRQ*.



Hình 32: Cấu trúc thanh ghi *NVIC_ISERx*.

- Để thuận tiện xử lý đối với số *IRQ* có vị trí ô thuộc về *NVIC_ISER1* và *NVIC_ISER2* thì ta nên chia lấy dư để đưa về khoảng vị trí từ 0 tới 31.
 - Đối với các số *IRQ* có vị trí ô thuộc về *NVIC_ISER1*:

$$1 \ll (\text{IRQNumber} \% 32)$$

Bảng 25: Công thức thiết lập giá trị với số *IRQ* trong *NVIC_ISER1*.

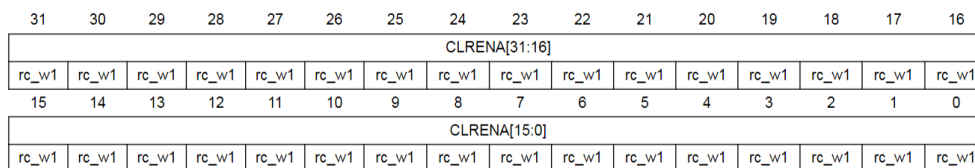
- Đối với các số *IRQ* có vị trí ô thuộc về *NVIC_ISER2*:

$$1 \ll (\text{IRQNumber} \% 64)$$

Bảng 26: Công thức thiết lập giá trị với số *IRQ* trong *NVIC_ISER2*.

14.2. Thanh ghi *NVIC_ICERx* (x từ 0 tới 2):

- Dùng để xóa thiết lập cho phép ngắt luồng theo số *IRQ*.



Hình 33: Cấu trúc thanh ghi *NVIC_ICERx*.

- Chỉ cho phép ghi và xóa đồng thời thông qua mức 1, các thao tác với mức 0 không có tác dụng.
- Khi ghi mức 1 trên thanh ghi sẽ hiểu là vô hiệu ngắt luồng.

- Khi đọc thanh ghi, mức 1 sẽ hiểu là ngắt luồng đang hoạt động và mức 0 là ngược lại.

14.3. Thanh ghi NVIC_ISPRx (x từ 0 tới 2):

- Dùng để ép buộc ngắt luồng vào trạng thái chờ và trả về trạng thái.
- Ghi mức 1 trên ô bit tương ứng có ngắt luồng đang ở trạng thái chờ sẽ không có tác dụng.
- Ghi mức 1 trên ô bit tương ứng có ngắt luồng bị vô hiệu sẽ chuyển ngắt luồng sang trạng thái chờ.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SETPEND[31:16]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SETPEND[15:0]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Hình 34: Cấu trúc thanh ghi NVIC_ISPRx.

14.4. Thanh ghi NVIC_ICPRx (x từ 0 tới 2):

- Dùng để xóa trạng thái chờ của ngắt luồng và trả về trạng thái.
- Chỉ cho phép ghi và xóa đồng thời thông qua mức 1, các thao tác với mức 0 không có tác dụng.
- Khi ghi mức 1 trên thanh ghi sẽ hiểu là xóa trạng thái chờ của ngắt luồng.
- Khi đọc thanh ghi, mức 1 sẽ hiểu là ngắt luồng đang trạng thái chờ và mức 0 là ngược lại.

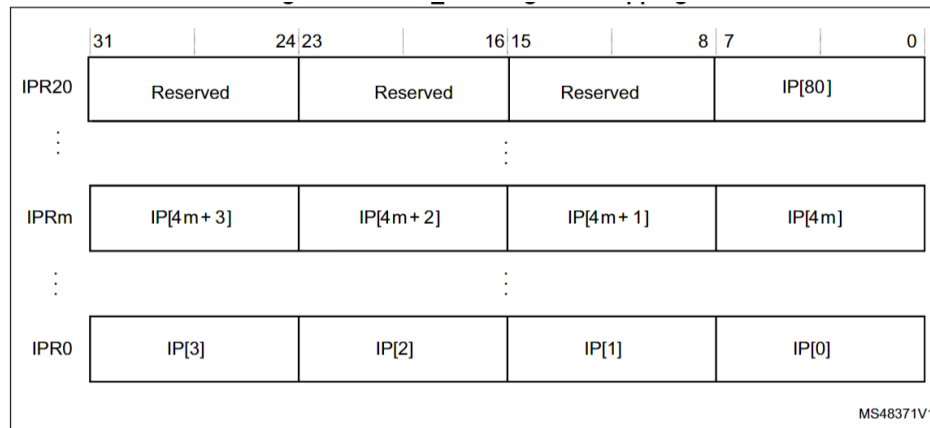
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLRPEND[31:16]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CLRPEND[15:0]															
rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Hình 35: Cấu trúc thanh ghi NVIC_ICPRx.

14.5. Thanh ghi NVIC_IPRx (x từ 0 tới 21):

- Dùng để lưu mức ưu tiên của ngắt luồng.

- Cho phép lưu ghép giữa mức ngắt ưu tiên (*Preemption priority*) và mức ngắt phụ (*Sub priority*) (phần này sẽ không được đề cập tới).



Hình 36: Cấu trúc thanh ghi NVIC_IPRx.

- Để tìm vị trí lưu cho số IRQ, ta có các công thức sau:
 - Số thứ tự của thanh ghi NVIC_IPR:

$$\text{Số thứ tự của thanh ghi} = \text{IRQNumber} / 4$$

Bảng 27: Công thức tính số thứ tự thanh ghi.

- Độ dời vị trí truy cập:

$$\text{Độ dời vị trí truy cập} = \text{IRQNumber} \% 4$$

Bảng 28: Công thức tính độ dời vị trí truy cập.

- Công thức truy cập cuối cùng:

$$\text{IRQPriority} \ll ((8 * \text{byte_offset}) + (8 - \text{NOT_IMPLEMENTED_OFFSSET}))$$

Bảng 29: Công thức truy cập cuối cùng.

- Trong đó, NOT_IMPLEMENTED_OFFSSET được quy định là 4-bit đầu tiên.

15. Cơ chế xử lý ngắt luồng đối với ngoại vi GPIO:

15.1. Ở phía ngoại vi:

- Do ngoại vi GPIO ở vi điều khiển không có các thanh ghi dành cho xử lý EXTI nên ta phải gọi tới ngoại vi AFIO trong đó chứa 4 thanh ghi cho phép xử lý là AFIO_EXTICR1, AFIO_EXTICR2, AFIO_EXTICR3 và AFIO_EXTICR4.

- Sau đó thực hiện, truy cập vào ô đại diện cho chân *I/O* cần gọi ngắt và điền giá trị mã hóa cho tên các cổng *GPIO*.
- Kế đến, thực hiện gỡ lớp phủ ngắt thông qua thanh ghi `EXTI_IMR`.
- Cuối cùng, thực hiện ghi mức 1 cho chân *I/O* trên thanh ghi `EXTI_PR`.
- Sau khi thực hiện xong ngắt luồng thì thực hiện ghi mức 1 cho chân *I/O* trên thanh ghi `EXTI_PR`.

15.2. Ở phía NVIC:

- Thực hiện reset 2 thanh ghi là `NVIC_ISER` và `NVIC_ICER` và điền mức 1 cho ô đại diện cho số *IRQ*.
- Thực hiện điền mức ưu tiên *IRQ* vào thanh ghi chứa ô đại diện cho số *IRQ*.

16. Tìm hiểu cấu hình xử lý ngắt luồng cho ngoại vi *GPIO*:

- Đối với hàm `GPIO_Init` đã được đề cập, sau khi thực hiện xong các nội dung ở trên, ta sẽ thực hiện khởi động xung nhịp ngoại vi *AFIO* và các thao tác như đoạn mã sau:

```
AFIO_PERI_CLK_ENB();
//Set trigger cạnh xuống
if (pin_falling_edge == 1) {
    EXTI->FTSR |= (1 << pin_number);
    EXTI->RTSR &= ~(1 << pin_number);
}
//Set trigger cạnh lên
if (pin_rising_edge == 1) {
    EXTI->RTSR |= (1 << pin_number);
    EXTI->FTSR &= ~(1 << pin_number);
}

uint8_t exti_line      = (pin_number / 4) + 1;
uint8_t port_position  = pin_number % 4;
uint8_t portcode       = (pGPIO_Handler->pGPIOx == GPIOA) ? 0 :
                          (pGPIO_Handler->pGPIOx == GPIOB) ? 1 :
                          (pGPIO_Handler->pGPIOx == GPIOC) ? 2 :
                          (pGPIO_Handler->pGPIOx == GPIOD) ? 3 :
```

```

        (pGPIO_Handler->pGPIOx == GPIOE) ? 4 :
        (pGPIO_Handler->pGPIOx == GPIOF) ? 5 : 6;
if (exti_line == 1) {
    //CLEAR BIT AND IMPORT VALUE
    AFIO->_EXTI1 &= ~(0xF << (port_position * 4));
    AFIO->_EXTI1 |= portcode << (port_position * 4);
} else if (exti_line == 2) {
    //CLEAR BIT AND IMPORT VALUE
    AFIO->_EXTI2 &= ~(0xF << (port_position * 4));
    AFIO->_EXTI2 |= portcode << (port_position * 4);
} else if (exti_line == 3) {
    //CLEAR BIT AND IMPORT VALUE
    AFIO->_EXTI3 &= ~(0xF << (port_position * 4));
    AFIO->_EXTI3 |= portcode << (port_position * 4);
} else if (exti_line == 4) {
    //CLEAR BIT AND IMPORT VALUE
    AFIO->_EXTI4 &= ~(0xF << (port_position * 4));
    AFIO->_EXTI4 |= portcode << (port_position * 4);
}
EXTI->_IMR &= 0x00000000;
EXTI->_IMR |= (1 << pin_number);
EXTI->_PR &= 0x00000000;
EXTI->_PR |= (1 << pin_number);

```

Bảng 30: Mở rộng hàm GPIO_Init.

- Hàm GPIO_IRQ_Interrupt_Config:
 - o Thực hiện xóa và kích hoạt ngắt luồng.

```

void GPIO_IRQ_Interrupt_Config(uint8_t IRQNumber, uint8_t ENorDI) {
    NVIC_ISER->ISER[0] &= 0x00000000;
    NVIC_ISER->ISER[1] &= 0x00000000;
    NVIC_ISER->ISER[2] &= 0x00000000;
    NVIC_ICER->ICER[0] &= 0x00000000;
    NVIC_ICER->ICER[1] &= 0x00000000;
    NVIC_ICER->ICER[2] &= 0x00000000;
    if (ENorDI == ENABLE) {
        //Dùng để kích hoạt ngắt luồng
    }
}

```

```

        if (IRQNumber <= 31) {
            NVIC_ISER->_ISER[0] |= (1 << IRQNumber);
        } else if (31 < IRQNumber && IRQNumber < 64) {
            NVIC_ISER->_ISER[1] |= (1 << (IRQNumber % 32));
        } else if (64 <= IRQNumber && IRQNumber < MAX_INTERRUPTS) {
            NVIC_ISER->_ISER[2] |= (1 << (IRQNumber % 64));
        }
    } else {
        //Dùng để vô hiệu ngắt luồng
        if (IRQNumber <= 31) {
            NVIC_ICER->_ICER[0] |= (1 << IRQNumber);
        } else if (31 < IRQNumber && IRQNumber < 64) {
            NVIC_ICER->_ICER[1] |= (1 << IRQNumber % 32);
        } else if (64 <= IRQNumber && IRQNumber < MAX_INTERRUPTS) {
            NVIC_ICER->_ICER[2] |= (1 << IRQNumber % 64);
        }
    }
}

```

Bảng 31: đoạn mã hàm GPIO_IRQ_Interrupt_Config.

- Hàm GPIO_IRQ_Number_Config:

- Thực hiện thiết lập đối với mức ưu tiên IRQ.

```

void GPIO_IRQ_Number_Config(uint8_t IRQNumber, uint8_t IRQPriority) {
    uint8_t ipr_number = IRQNumber / 4;
    uint8_t byte_offset = IRQNumber % 4;
    NVIC_IPR->_IPR[ipr_number] |=
    (IRQPriority << ( (8 * byte_offset) + (8 - NOT_IMPLEMENTED_OFFSET) ));
}

```

Bảng 32: đoạn mã hàm GPIO_IRQ_Number_Config.

- Hàm GPIO_IRQ_Handler:

- Thực hiện xử lý sau khi kết thúc nội dung ngắt.

```

void GPIO_IRQ_Handler(uint8_t PinNumber) {
    if (EXTI->_PR & (1 << PinNumber)) {
        EXTI->_PR |= (1 << PinNumber);
    }
}

```



```

    }
}

```

Bảng 33: đoạn mã hàm *GPIO_IRQ_Handler*.

- Thực hiện override các hàm xử lý *ISR – Interrupt Service Routine* (các hàm được định nghĩa sẵn không có kiểu trả về là đầu vào là kiểu void khi khởi động chương trình và cho phép override, thường sẽ có đuôi là *IRQHandler* trong tên hàm).
 - o Ví dụ, giả sử khi có xử lý ngắt luồng ở line 6 của chân *I/O* có công bất kỳ và thực hiện các hàm cấu hình đầy đủ, ta sẽ override lại hàm xử lý (hàm *Handler*) như sau:

```

void EXTI9_5_IRQHandler(void) {
    GPIO_IRQ_Handler(*số thứ tự của chân I/O*);
    *nội dung thực hiện ngắt*
}

```

Bảng 34: Ví dụ override hàm xử lý *ISR*.

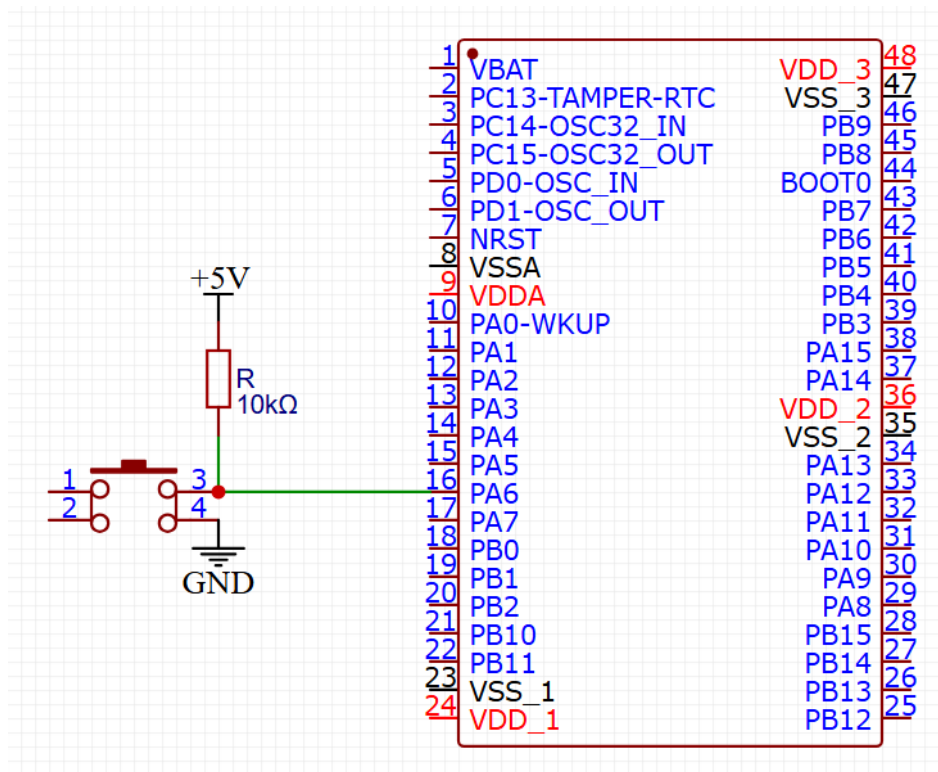
17. Kiểm định bằng các chương trình mẫu:

17.1. Điều khiển LED bằng nút nhấn ngoài có nối trở 10kOhm:

17.1.1. Thiết bị sử dụng:

- Bộ vi điều khiển *STM32F103C8T6*.
- Mạch nạp *USB-A mini ST-LINKv2*.
- Breadboard 62x10 kèm 50x4 đường dẫn pha nóng – lạnh.
- Nút nhấn 4 chân 12x12x12mm.
- 3 dây đực-đực.
- Điện trở 10kOhm.

17.1.2.Sơ đồ kết nối:



Hình 37: Sơ đồ kết nối chương trình 1.

17.1.3.Lập trình cấu hình:

- PC13 sẽ được cấu hình là LED (LED hàn sẵn trên mạch).
- PA6 sẽ được cấu hình là chân điều khiển.
- Đoạn mã cấu hình:

```
GPIO_HANDLER gpiopc13led;  
gpiopc13led.pGPIOx = GPIOC;  
gpiopc13led.GPIO_Pin_Config.__PIN_NUMBER      = GPIO_PIN_NUM_13;  
gpiopc13led.GPIO_Pin_Config.__PIN_CFG_TYPE     = GPIO_CNF_PP_OUT;  
gpiopc13led.GPIO_Pin_Config.__PIN_MODE        = GPIO_MODE_OUT_50MHz;  
GPIO_Clock_Control(gpiopc13led.pGPIOx, ENABLE);  
GPIO_Init(&gpiopc13led);  
GPIO_Write_To_Pin  
(gpiopc13led.pGPIOx, gpiopc13led.GPIO_Pin_Config.__PIN_NUMBER, GPIO_PIN_UNSET);  
GPIO_HANDLER gpiopa6button;
```

```

gpiopa6button.pGPIOx = GPIOA;
gpiopa6button.GPIO_Pin_Config.__PIN_NUMBER          = GPIO_PIN_NUM_6;
gpiopa6button.GPIO_Pin_Config.__PIN_CFG_TYPE        = GPIO_CNF_PUPD_IN;
gpiopa6button.GPIO_Pin_Config.__PIN_MODE            = GPIO_MODE_IN;
//gpiopa6button.GPIO_Pin_Config.__PIN_PU_PD         = true;
//gpiopa6button.GPIO_Pin_Config.__PIN_FE_T          = true;
GPIO_Clock_Control(gpiopa6button.pGPIOx, ENABLE);
GPIO_Init(&gpiopa6button);

```

Bảng 35: đoạn mã cấu hình chương trình 1.

- Thuật toán:
 - Tạo ra vòng lặp vô hạn.
 - Đọc trạng thái nút nhấn.
 - Nếu trạng thái nút nhấn bằng 0:
 - Delay bằng vòng lặp.
 - Bật đèn LED.
 - Ngược lại:
 - Delay bằng vòng lặp.
 - Tắt đèn LED.
 - Đoạn mã sử dụng:

```

while(1) {
    uint8_t status_button = GPIO_Read_From_Pin(gpiopa6button.pGPIOx,
gpiopa6button.GPIO_Pin_Config.__PIN_NUMBER);
    if (status_button == 0) {
        delay();
        GPIO_Write_To_Pin(gpiopc13led.pGPIOx,
gpiopc13led.GPIO_Pin_Config.__PIN_NUMBER, GPIO_PIN_SET);
    } else {
        delay();
        GPIO_Write_To_Pin(gpiopc13led.pGPIOx,
gpiopc13led.GPIO_Pin_Config.__PIN_NUMBER, GPIO_PIN_UNSET);
    }
}

```

Bảng 36: đoạn mã thuật toán chương trình 1.

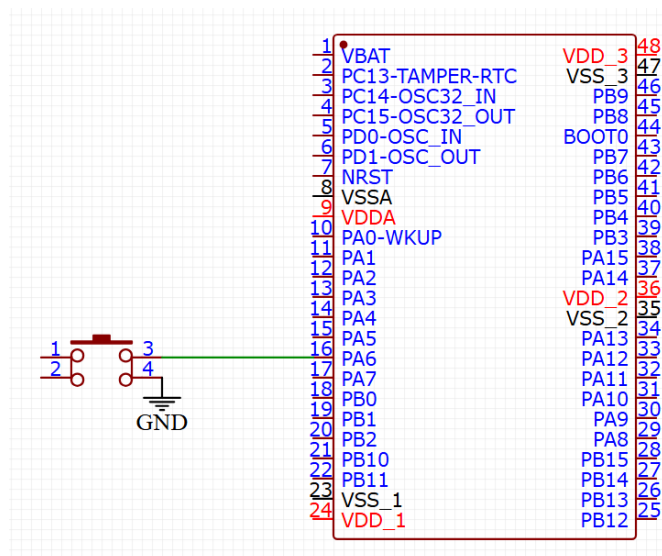
- Kết quả từ báo cáo viên: Thành công, không xảy ra lỗi.

17.2. Điều khiển LED bằng nút nhấn ngoài không có trở và dùng ngắt ngoài:

17.2.1.Thiết bị sử dụng:

- Bộ vi điều khiển *STM32F103C8T6*.
- Mạch nạp *USB-A mini ST-LINKv2*.
- Breadboard 62x10 kèm 50x4 đường dẫn pha nóng – lạnh.
- Nút nhấn 4 chân 12x12x12mm.
- 2 dây đực-đực.

17.2.2.Sơ đồ kết nối:



Hình 38: Sơ đồ kết nối chương trình 2.

17.2.3.Lập trình cấu hình:

- Tương tự so với chương trình mẫu thứ nhất nhưng nút bấm được cấu hình thêm biến ghi nhận loại chuyển cạnh.
- Thuật toán:
 - o Gọi ra hàm cấu hình cho ngắt luồng và mức ưu tiên *IRQ*.
 - o Tạo vòng lặp vô hạn.
 - o Cấu hình hàm xử lý *ISR* bên dưới chương trình chính.

- Khi chương trình đang trong vòng lặp vô hạn, nếu bất kỳ tín hiệu nào phát ra từ nút nhấn sẽ lập tức bật đèn LED và sẽ tắt trong lần nhấn kế tiếp.
- Đoạn mã sử dụng:

```
int main(void) {
    *đoạn mã cấu hình*
    GPIO_IRQ_Interrupt_Config(IRQ_NUM_EXTI_9_TO_5, ENABLE);
    GPIO_IRQ_Number_Config(IRQ_NUM_EXTI_9_TO_5, 1);
    While (1);
}
void EXTI9_5_IRQHandler(void) {
    GPIO_IRQ_Handler(GPIO_PIN_NUM_6);
    GPIO_Toggle_Pin(GPIOC, GPIO_PIN_NUM_13);
}
```

Bảng 37: đoạn mã thuật toán chương trình 2.

- Kết quả từ báo cáo viên: Thành công, nhưng xảy ra lỗi khi lắp thêm điện trở 10kOhm vào (chưa giải quyết được).

CHƯƠNG 3: KẾT LUẬN

- Đã hoàn thiện 95% trình điều khiển cho ngoại vi GPIO (kể cả xử lý ngắt ngoài) tuy nhiên chưa hoàn thiện một số chức năng phụ.
- Hoàn thiện báo cáo tương đối đầy đủ về mặt kiến thức, chỉnh chu về mặt hình thức.
- Nắm chắc cách sử dụng các tài liệu phụ trợ của nhà sản xuất đối với vi điều khiển, khai thác sử dụng đa dạng nguồn thông tin để xây dựng trình điều khiển.

TÀI LIỆU THAM KHẢO

- [1] A. Limited, ARM® Cortex®-M3 Processor Technical Reference Manual, ARM Limited, (2005-2008).
- [2] A. Limited, ARM ® v7-M Architecture Reference Manual, ARM Limited, 2006-2008.
- [3] STMicroelectronics, STM32F10xxx/20xxx/21xxx/L1xxxx Cortex ® -M3

programming manual, STMicroelectronics, 2009-2017.

- [4] STMicroelectronics, STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs Reference Manual, STMicroelectronics, 2007-2008.
- [5] STMicroelectronics, STM32F103x6 STM32F103x8 STM32F103xB Datasheet, STMicroelectronics, 2007.
- [6] K. Nayak, *Learn bare-metal driver development using Embedded C : Writing drivers for STM32 GPIO,I2C, SPI,USART from scratch*, FastBit Embedded.